Ops — compiled path

> Compiled from `/Users/darkonikolic/Documents/learning/paths/Ops`. Canonical sources stay in place.

Phase — 01-overview-and-scope

Directory: `01-overview-and-scope/`

Source: `01-overview-and-scope/01-program-architecture-thinking-and-roadmap.md`

Ops / Platform / Cloud — mindset and phased roadmap

This path supports **hands-on comprehension** of how modern backends are **built**, **tested**, **packaged**, **distributed**, **deployed**, **observed**, **maintained**, **incident-handled**, and **safely improved** — not trivia memorised separately from systemic behaviour.

What deep understanding means here

How an artefact survives from source control curiosity into repeatable production behaviours: pipelines, artefacts, rollout surfaces, observable runtime, corrective action, guarded evolution.

Area map (numeric order = topic progression)

```
01  Overview & scope
── FUNDAMENT ──────────────────────────────────────────────────────────
02  Linux administration        Host triage, bash scripting, systemd, ssh
03  Shell scripting for ops     set -euo pipefail, jq, yq, idempotency, trap, retry
04  Python for ops              boto3, subprocess, requests, click, K8s client
05  Git & release workflow      Branches, merges, tags, recovery
06  Network flow                DNS, TLS, NAT, routing, firewall
── CONTAINERS ─────────────────────────────────────────────────────────
07  Docker                      Images, Dockerfile, compose, multi-stage, hardening
08  Podman + Quadlets           Rootless runtime, systemd-native Quadlet units
09  Container networking        Bridges, DNS, NAT, inter-service communication
── KUBERNETES & HELM ──────────────────────────────────────────────────
10  Kubernetes                  Workloads, RBAC, networking, storage, ingress, ops
11  Helm                        Charts, values, lifecycle, OCI registry, GitOps
── INFRASTRUCTURE AS CODE ─────────────────────────────────────────────
12  Terraform + LocalStack + CDK  State, modules, drift, local AWS testing, CDK intro
13  Ansible                     Playbooks, roles, vault, dynamic inventory, EC2 fleet
── CI/CD & DELIVERY ───────────────────────────────────────────────────
14  GitLab CI/CD                Pipelines, runners, docker build, gates, deploy
15  DevSecOps / supply chain    Scanners, SBOM, SAST, secret scan, pipeline gates
16  Production reliability      Blue/green, canary, rollbacks, DR, postmortem
── AWS ────────────────────────────────────────────────────────────────
17  AWS                         IAM, VPC, EC2, RDS, S3, ALB, ECS Fargate, EKS,
                                CloudFront, WAF, Secrets, DR + Fargate lab +
                                CodePipeline/CodeDeploy
── OBSERVABILITY ──────────────────────────────────────────────────────
18  Edge / reverse proxy        Nginx, Traefik, TLS termination, LB, traffic shaping
19  Prometheus & PromQL         Golden signals, scrape, exporters, alerting
20  Loki & logging              Structured logs, correlation, forensic discipline
21  OpenTelemetry & tracing     Spans, context propagation, sampling, collector
22  Unified observability       Grafana multi-signal, SLOs, error budgets, incidents
23  Secrets management          Vault, K8s secrets, rotation, CI variable hygiene
24  Incident troubleshooting    K8s, networking, datastore, pipeline scenario labs
25  Performance optimization    Profiling, bottleneck triage, load testing
26  Platform engineering        IDPs, golden paths, self-service, paved roads
```

Source: `01-overview-and-scope/02-understand-implement-break-fix-document-loop.md`

Practise discipline — intentional fault injection and documentation

Rule of the entire path — theory grounded in breakage

Standalone theory consumption is insufficient: for each competency cluster you practise the loop:

Understand the concept → implement it cleanly → deliberately fault the system → **interpret what telemetry and symptoms mean** → repair with controlled reversibility → **document hypotheses, evidence chain, corrective steps, rollback notes**, plus follow-ups that harden recurrence probability downward.

Operational mindset aspiration:

Listen for what the infrastructure is trying to imply — not vague “broken” vibes without inspectable artefacts (logs, service states, resource budgets, timelines).

Source: `01-overview-and-scope/03-required-practice-surfaces-linux-git-phases.md`

Required practice surfaces — Linux stack through platform engineering arcs

Operate only machines and tenants you ethically control.

Linux labs: authorised Ubuntu VMs, disposable directories, scripted reversible failures, reproducible notes (never paste secrets).

Git / GitLab labs: Symfony (or analogous) codebase plus local GitLab (or compliant equivalent) for merge/release rehearsal without customer data leakage.

Downstream corridors (`04` … `23`)

Area	Typical lab ingredients *(adapt to local policy)*
CI/CD GitLab (`04-*`)	`gitlab-runner`, Docker-capable runners, PHPUnit/Composer, PHPStan/static gates, pipeline secret hygiene
Docker (`05-*`)	Docker Engine + Compose plugin + Symfony mocks; trailing units (`11-*` **onward**) deepen **supply-chain & hardening posture** responsibly
Podman (`06-*`)	`podman`, compose compatibility quirks, systemd unit generation
Container networking (`07-*`)	User-defined bridges, internal DNS drills, constrained packet capture tuition
Kubernetes (`08-*`)	Lightweight clusters (**k3d**, kind…), disposable playgrounds; trailing units broaden **capacity, PDB, quotas, NetworkPolicies** ethically
Helm (`09-*`)	Charts on lab clusters; trailing units practise **schemas, hooks/tests, OCI registries, GitOps hand-off language** consciously
Terraform / IaC (`10-*`, `19-*`)	Terraform/OpenTofu + AWS learner accounts responsibly; `10-*` **trailing worksheets** widen backend/apply safety; `19-*` extends AWS breadth
App networking foundations (`11-*`)	Host `nginx`, `openssl`, `dig`, `ss`, cautious `tcpdump`
Edge proxies & shaping (`12-*`)	Nginx ↔ Traefik, TLS offload, timeouts, selective compression, sane rate limits
Monitoring (`13-*`)	Prometheus, exporters, PromQL playgrounds, alerting hooks
Logging (`14-*`)	Grafana Loki ingestion stack, correlation IDs across Symfony + Go
Tracing (`15-*`)	OpenTelemetry Collector (+ Tempo/Jaeger-class backends sized for labs)
Observability synthesis (`16-*`)	Grafana boards merging metrics/logs/traces
Secrets (`17-*`)	Vault OSS sandbox optional · GitLab protected variables · K8s Secret realism
DevSecOps (`18-*`)	Trivy, SBOM, SAST/additional scanners in CI
AWS operations (`19-*`)	Core IAM/VPC/compute/storage/ECS/EKS introductions; `15-*` **onward** previews org networking, ECS vs EKS trade space, edges, resilience & cost framing—ethical learner accounts only
Reliability rituals (`20-*`)	Rollbacks, backup/restore rehearsals, rollout strategy narratives
Incident labs (`21-*`)	Controlled chaos across cluster, nets, pipelines, datastore edges
Performance (`22-*`)	Symfony Profiler, Go `pprof`, Redis caches, Postgres plans, synthetic load tooling
Platform engineering arcs (`23-*`)	Golden paths templates summarising repeatable paved roads

Reference materials *(verify listings stay current)*

Track	Surface	Guidance
Linux starter	LinuxJourney	Core modules reinforcing shell/systemd/logs
Linux course	Marketplace “Linux Administration Bootcamp” (titles vary)	Skip enterprise-only tangents unless you branch deliberately
Docker + Kubernetes practical course	Frequently titled **“Docker & Kubernetes: Practical Guide”** style offerings	Fits Docker → networking → Kubernetes → partial Helm scaffolding; deepen AWS/observability here in vault worksheets
K8s exam-prep supplements	KodeKloud CKA flavours / similar	Disposable clusters—not production tenants
Observability backends	Grafana, Prometheus, Loki, OTLP Collector, Jaeger/Tempo docs	Compose minimal stacks; watch resource burn
Scanners	Trivy, optional Syft/Grype, GitLab Ultimate security tiers if licensed	Tune severities thoughtfully
Performance	Symfony Profiler docs, Go `pprof` guide, Postgres performance primers	Cross-link `MySQL-Database-Engineering` parallels for relational debugging habits
AWS	Official Foundations + IAM guides + Well-Architected primer	Honour spend alarms + least-privilege rehearsals

Baseline toolchain hint (Ubuntu-oriented)

After `sudo apt update`, habitual helpers often include `htop`, `tmux`, `jq`, `curl`, `rsync`, `tree`, `vim`; reconcile `net-tools` nostalgia vs `ip`/`ss` modern defaults consciously.

Maintain a lab workspace such as `opslab/` for reproducible scripted exercises and incident notebooks scrubbed of secrets.

Phase — 02-linux-administration-server-mindset

Directory: `02-linux-administration-server-mindset/`

Source: `02-linux-administration-server-mindset/01-filesystem-cli-and-hierarchy-mind-model.md`

Unit 01 — Filesystem posture, hierarchy literacy, habitual CLI navigation

Comfortable primitives: `pwd`, `ls`, `cd`, `mkdir`, `rm`, `mv`, `cp`, `touch`, directory trees (tooling like `tree` when installed), shell history ergonomics (`history`, purposeful clearing rituals — never erase audit trails accidentally on shared infra).

Labs:

```
mkdir -p app logs backup
touch nginx.log db.log
cp nginx.log backup/
mv db.log logs/
```

Mental anchors

Interpret **absolute versus relative paths** consciously when scripting — surprises often trace back ambiguous `cwd`.

Locate standard hierarchy anchors (`/etc`, `/var`, `/tmp`, `/home`, `/usr`, …) and articulate what classes of mutable vs configuration-of-record artefacts typically reside where.

Source: `02-linux-administration-server-mindset/02-users-permissions-privilege-thinking.md`

Unit 02 — Users, ownership, permission bits, audited privilege escalation

Tools: `chmod`, `chown`, `groups`, `sudo`, `whoami`, `id`.

Interpret `-rwxr-xr-x` triplets distinctly for owners, groups, and others—note differing semantics **files vs directories** (execute on directory ≈ traversal / lookup permission).

Controlled experiment:

```
touch secret.txt
chmod 600 secret.txt
chmod 755 deploy.sh   # when ethically present inside disposable lab artefacts
sudo useradd -m testuser   # later remove cleanly with symmetrical housekeeping
```

Mindset emphasis

Incidents originate often from unintended ownership drift, brittle `sudoers` expectations, careless world-readable sensitive paths —practice reading listings like calm pager introspection rehearsals.

Source: `02-linux-administration-server-mindset/03-process-visibility-signalling-and-daemons.md`

Unit 03 — Process visibility, signalling patterns, graceful versus abrupt termination

Comfortable ergonomics: `ps aux`, `top`, `htop`, `kill`, `killall`, job introspection (`jobs`), `bg`, `fg`.

Guided lab

Spawn a deliberately long idle footprint:

```
sleep 500 &
```

Locate PID, validate characteristics in tooling, rehearse escalating signals — prefer cooperative shutdown pathways before abrupt escalation when ethically acceptable locally.

Articulate distinctions between systemd-supervised daemon lifecycles vs ephemeral shell-managed jobs disappearing on session collapse.

Source: `02-linux-administration-server-mindset/04-systemd-and-journal-correlation.md`

Unit 04 — systemd unit literacy plus correlated journalling ingestion

Operational verbs spanning enable/disable choreography, restrained `restart` vs `reload` where documentation differentiates behavioural impact on long-lived sockets.

Inspect aggregates:

```
journalctl -xe
journalctl -u ssh --since "15 min ago"
```

Break-fix rehearsal ethical constraint

Stopping remoting daemons blindly risks lockout — coordinate safeguards (console access, ephemeral VM, reversible snapshots) beforehand.

Source: `02-linux-administration-server-mindset/05-package-management-apt-and-dpkg-thinking.md`

Unit 05 — APT / dpkg ergonomics plus dependency humility

Exercise package lifecycle ethically on ephemeral hosts—for example reversible nginx experimentation when policy aligns; alternatively choose smaller CLI payloads.

Mindful sequences:

```
sudo apt install …
sudo apt remove …
dpkg -l | head                  # illustrative sampling only
```

Mindset rehearsals

Enumerate repository trust sourcing, versioning drift, leftover configuration directories (`purge` ramifications), orphaned dependency fallout, destructive `autoremove` blindness—simulate planning before irreversible housekeeping.

Source: `02-linux-administration-server-mindset/06-ssh-scp-rsync-and-tmux-mindset.md`

Unit 06 — SSH identity, multiplexed sessions, file exchange ergonomics

`ssh-keygen` with conscientious passphrase and agent ergonomics—rehearse sanely inside laboratories reflecting eventual workplace constraints.

Transfers: exploratory `scp` hops plus meticulous `rsync` passes scrutinising traversal semantics (`/` suffix discipline, reciprocal source/destination reciprocity pitfalls).

Operate `tmux` (alternatively classical `screen`) demonstrating detach ↔ reattach resilience against flaky transports—articulate unattended shell pitfalls and jump-host privilege bleed cautions.

Source: `02-linux-administration-server-mindset/07-bash-scripting-automation-habits.md`

Unit 07 — Bash iteration, branching, guard-railed scripting habits

Iteration illustration:

```
#!/usr/bin/env bash
for f in *.log; do
  grep ERROR "$f" || true
done
```

Author `backup.sh` compressing rotations, relocating generations, pruning aged snapshots—with dry rehearsals before trusting destructive trims.

Articulate conscientious quoting, purposeful exit signalling, restrained `errexit` adoption balancing diagnostics friendliness organisational norms prescribe.

Source: `02-linux-administration-server-mindset/08-cron-and-non-interactive-pitfalls.md`

Unit 08 — Cron discipline and silent-environment failure archaeology

`crontab -e` invokes `backup.sh` on a disciplined cadence (example: five-minute iteration only after validating resource impact ethically).

Assume reduced inherited environment semantics—explicit absolute paths, restrained reliance on personalised shell aliases, logging stdout/stderr to inspectable artefacts by design rather than drowning silently inside `MAILTO` oblivion unintentionally configured away.

Source: `02-linux-administration-server-mindset/09-logs-resources-and-guided-fault-remediation.md`

Unit 09 — Correlate logs with disk, sockets, memory, and scheduler artefacts

Fluent use of `tail` / `less`, precise `grep`, `journalctl` windows, `ss` (supplement nostalgic `netstat` stories only where legacy estates demand), `df` / `du` / `free`, `uptime` posture.

Controlled fault recipes you author consciously: depleted disk exhaustion, extinguished ancillary unit, brittle permissions starving automation, brittle `cron` path assumptions—recover while narrating reproducible timelines (hypothesis → evidence → corrective diff → safeguards).

Source: `02-linux-administration-server-mindset/10-linux-integration-autonomous-incident-cycle.md`

Unit 10 — Blind rehearsal plus nginx-centric break/recover narration

Navigate without reference crutches initially: correlate symptoms ↔ authoritative logs ↔ measured restarts versus deeper remediation; rectify permission drift hampering scripted operators; reaffirm sane process inventories; scrutinise RAM/disk budgets; dissect scheduler fidelity; reaffirm unaffected SSH ergonomics responsibly.

Laboratory climax: operate nginx—or lighter analogue aligning with organisational policy—and choreograph degrade → recover journaling emphasising deterministic reasoning rather than ad hoc button mashing.

Capture concise artefacts redacting privileged tokens—articulating causal class, illuminating log excerpts corroboratively, enumerated remediation steps plus preventative deltas.

Phase — 03-shell-scripting-for-ops

Directory: `03-shell-scripting-for-ops/`

Source: `03-shell-scripting-for-ops/01-script-skeleton-and-anatomy.md`

Shell — `01` Script skeleton i anatomija

Zasto: Svaka skripta koju napišeš za ops mora imati isti osnov — shebang, zaštitne flagove, main funkciju i smislene exit kodove. Bez toga pišeš skriptu koja tiho kvari produkciju, radi lokalno ali ne u CIu, ili ostavlja smeće kad pukne.

Minimalni šablon koji uvijek koristiš

```
#!/usr/bin/env bash
set -euo pipefail

# Konstante
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly SCRIPT_NAME="$(basename "$0")"

main() {
  echo "skripta radi"
}

main "$@"
```

Svaka linija ima razlog:

`#!/usr/bin/env bash` — traži bash kroz PATH umjesto hardkodovanog `/bin/bash`. Na Macu je `/bin/bash` stari bash 3.x, na serveru može biti drugačiji. `env bash` uvijek nađe pravi.

`set -e` — izlazi odmah kad neka komanda vrati non-zero. Bez ovoga skripta nastavlja izvršavanje nakon greške i može obrisati pogrešnu bazu, deployati loš image, ili preskočiti kritičan korak.

`set -u` — nedefenisana varijabla je greška, ne prazan string. Bez ovoga `rm -rf /$TYPO/` briše root. Sa ovime padne s jasnom greškom.

`set -o pipefail` — pipeline `a | b | c` vraća exit kod prvog koji nije uspio, ne zadnjeg. Bez ovoga `false | true` prolazi. Sa ovime pada.

`main "$@"` — sve funkcije definišeš iznad, poziv na dnu. `"$@"` prenosi sve argumente koji su proslijeđeni skripti.

Exit kodovi

Svaka skripta mora završiti sa smislenim exit kodom — CI i monitoring to čitaju.

```
main() {
  if ! check_dependency "docker"; then
    echo "ERROR: docker nije instaliran" >&2
    exit 1
  fi

  deploy || exit 2
}

check_dependency() {
  command -v "$1" &>/dev/null
}
```

Konvencija: - `0` — uspjeh - `1` — generalna greška - `2` — greška konfiguracije / nedostaje dependency - `3+` — specifični scenariji koje dokumentuješ na vrhu skripte

Greške uvijek idu na **stderr** (`>&2`), ne na stdout. Stdout je za output koji CI i drugi procesi konzumiraju — greške ga ne smiju zagaditi.

Cesta greška: set -e i neke komande legalno vraćaju non-zero

```
# OVO PADA — grep vraća 1 ako nema match
if grep "error" logfile; then ...

# ISPRAVNO
if grep -q "error" logfile; then ...

# ILI — eksplicitno dozvoli non-zero
grep "error" logfile || true
```

Vjezba

Napiši skriptu `check-services.sh` sa ovim šablonom: - Prima listu servisa kao argumente: `./check-services.sh nginx postgresql redis` - Za svaki servis provjerava da li je `systemctl is-active` vratio 0 - Ispisuje `[OK] nginx` ili `[FAIL] postgresql` - Izlazi sa kodom 1 ako ijedan servis nije aktivan, 0 ako su svi aktivni - Koristi `main "$@"`, sve u funkcijama, greške na stderr

---

Source: `03-shell-scripting-for-ops/02-variables-env-and-quoting.md`

Shell — `02` Varijable, env i quoting

Zasto: Greške u quotingu i rukovanju varijablama su broj jedan uzrok pucanja skripti u CI. Ime fajla sa razmakom, varijabla koja nije postavljena, PATH koji ne postoji u cron okruženju — sve to ubija skriptu tiho ili na spektakularan način.

Quoting — zlatno pravilo

Uvijek stavljaj varijable u navodnike. Bez navodnika bash radi word splitting i glob ekspanziju, što ima neočekivane efekte.

```
# Bez navodnika — pukne ako ime fajla ima razmak
filename="moj fajl.txt"
rm $filename       # bash vidi: rm moj fajl.txt → briše "moj" i "fajl.txt"

# Sa navodnicima — ispravno
rm "$filename"     # bash vidi: rm "moj fajl.txt"

# Globbing problem
pattern="*.txt"
ls $pattern        # bash ekspanduje *.txt OVDJE, ne prosljeđuje string komandi
ls "$pattern"      # prosljeđuje literal "*.txt"
```

Čitanje env varijabli

Tri načina, svaki za drugi slučaj:

```
# 1. Obavezna varijabla — pada s greškom ako nije postavljena
DB_HOST="${DB_HOST:?ERROR: DB_HOST mora biti postavljen}"

# 2. Opcionalna s defaultom — tiho koristi default
APP_PORT="${APP_PORT:-8080}"
APP_ENV="${APP_ENV:-development}"

# 3. Prosto čitanje — prazno ako nije postavljeno (rijetko što hoćeš u ops)
OPTIONAL_FLAG="${OPTIONAL_FLAG:-}"
```

`:?` je tvoj prijatelj za skripte koje deployuju. Ako zaboraviš postaviti `DB_HOST`, skripta odmah pada s jasnom porukom — ne deployuje na pogrešnu bazu.

Validacija na početku skripte

Sve varijable validiraj na vrhu, prije nego uradiš bilo šta:

```
main() {
  # Validacija — skripta pada ovdje, ne na pola deploya
  : "${APP_ENV:?}"
  : "${IMAGE_TAG:?}"
  : "${REGISTRY_URL:?}"

  # Defaults za opcionalne
  local dry_run="${DRY_RUN:-false}"
  local timeout="${DEPLOY_TIMEOUT:-120}"

  deploy "$APP_ENV" "$IMAGE_TAG"
}
```

Command substitution

```
# Zastarjelo — backticks, teško za čitanje i ne može se nestovati
VERSION=`git describe --tags`

# Moderno — $() sintaksa
VERSION=$(git describe --tags)

# Može se nestovati
SHORT_SHA=$(git rev-parse --short "$(git rev-parse HEAD)")

# Uvijek u navodnicima kad dodjeljuješ
HOSTNAME="$(hostname -f)"
```

Korisne string operacije bez spawnanja procesa

```
IMAGE="registry.example.com/myapp:v1.2.3"

# Ukloni prefiks do zadnje /
NAME="${IMAGE##*/}"          # → "myapp:v1.2.3"

# Ukloni sufiks od zadnje :
REPO="${IMAGE%:*}"           # → "registry.example.com/myapp"

# Uzmi samo tag
TAG="${IMAGE##*:}"           # → "v1.2.3"

# Zamjena
CLEAN="${IMAGE/registry.example.com\//}"  # → "myapp:v1.2.3"
```

Ovo je brže od `sed` za jednostavne transformacije jer ne spawna subprocess.

Export — šta child procesi nasljeđuju

```
# Lokalna varijabla — samo u trenutnoj ljusci
APP_ENV="staging"

# Exported — vidljiva child procesima (docker, kubectl, make...)
export APP_ENV="staging"

# Postavi i exportuj odjednom
export DB_HOST="db.internal"

# Provjeri što je exported
export -p | grep APP
```

Sve što CLI alati trebaju čitati mora biti exported.

Ceste greške

```
# GREŠKA — varijabla bez navodnika u if
if [ $status = "ok" ]; then    # pada ako je status prazan

# ISPRAVNO
if [ "$status" = "ok" ]; then

# GREŠKA — aritmetika sa stringovima
count=$count+1                 # "5+1", ne 6

# ISPRAVNO
count=$(( count + 1 ))
```

Vjezba

Napiši skriptu `deploy-validate.sh` : - Čita iz env: `APP_ENV` (obavezno), `IMAGE_TAG` (obavezno), `REPLICAS` (default: 2), `DRY_RUN` (default: false) - Validira da `APP_ENV` je jedan od: `staging` , `production` — ako nije, ispiši grešku i izađi s kodom 2 - Iz `IMAGE_TAG` (format: `registry/app:v1.2.3`) ekstraktuj samo verziju (`v1.2.3`) bez pozivanja `sed` ili `cut` - Ispisi sve vrijednosti na stdout u formatu `KEY=VALUE` , jednu po liniji

---

Source: `03-shell-scripting-for-ops/03-control-flow-and-functions.md`

Shell — `03` Control flow i funkcije

Zasto: Uvjetna logika i funkcije su razlika između skripte koja radi jednu stvar i skripte koja se može koristiti u više scenarija. U opsu to znači: isti kod za staging i production, isti deploy script za K8s i ECS.

---

if i testovi

Koristi `[[ ]]` umjesto `[ ]` u bash skriptama — nema word splitting, podržava `&&` / `||` i regex.

```
# Provjera fajla
if [[ -f "/etc/nginx/nginx.conf" ]]; then
  echo "nginx config postoji"
fi

# -f fajl postoji    -d direktorij    -e postoji (bilo što)
# -r čitljivo        -w zapisivo      -x izvršivo
# -s neprazan fajl   -L symlink

# String provjere
if [[ -z "$VAR" ]]; then      # prazan string
if [[ -n "$VAR" ]]; then      # neprazan string
if [[ "$ENV" == "prod" ]]; then
if [[ "$ENV" != "prod" ]]; then
if [[ "$ENV" =~ ^(staging|prod)$ ]]; then   # regex match

# Numeričke provjere
if (( count > 0 )); then      # aritmetički context, čistiji za brojeve
if [[ $count -gt 0 ]]; then   # tradicionalno, radi sa -eq -ne -lt -gt -le -ge
```

for i while

```
# Lista servisa
for service in nginx postgresql redis; do
  systemctl is-active --quiet "$service" || echo "FAIL: $service nije aktivan"
done

# Fajlovi u direktoriju (nikad ne parsuj ls!)
for logfile in /var/log/app/*.log; do
  [[ -f "$logfile" ]] || continue    # provjeri da glob nije ostao literal
  process_log "$logfile"
done

# Čitanje fajla liniju po liniju (jedini siguran način)
while IFS= read -r line; do
  echo "Processing: $line"
done < hosts.txt

# Čitanje outputa komande
while IFS= read -r pod; do
  kubectl logs "$pod" --tail=100
done < <(kubectl get pods -o name)

# Numerička petlja
for i in $(seq 1 5); do
  echo "Attempt $i"
  deploy && break || sleep $((i * 5))
done
```

case — za routing po env/tipu

```
case "$APP_ENV" in
  staging)
    REPLICAS=1
    DB_HOST="db-staging.internal"
    ;;
  production)
    REPLICAS=3
    DB_HOST="db-prod.internal"
    ;;
  *)
    echo "ERROR: nepoznat env '$APP_ENV'" >&2
    exit 2
    ;;
esac
```

Funkcije — jedini način da skripta ostane čitljiva

```
#!/usr/bin/env bash
set -euo pipefail

# Pravila:
# 1. Svaka funkcija radi jednu stvar
# 2. local za sve lokalne varijable — nikad ne zagađuj global scope
# 3. Funkcija komunicira kroz exit kod (0=ok, non-zero=greška) i stdout za vrijednosti

check_prereqs() {
  local missing=0
  for cmd in docker kubectl jq; do
    if ! command -v "$cmd" &>/dev/null; then
      echo "ERROR: '$cmd' nije instaliran" >&2
      missing=1
    fi
  done
  return $missing
}

get_current_image() {
  local deployment="$1"
  local namespace="${2:-default}"
  kubectl get deployment "$deployment" -n "$namespace" \
    -o jsonpath='{.spec.template.spec.containers[0].image}'
}

deploy() {
  local env="$1"
  local image_tag="$2"
  echo "Deploying $image_tag na $env..."
  # ...
}

main() {
  check_prereqs || exit 2

  local current
  current=$(get_current_image "myapp" "production")
  echo "Trenutni image: $current"

  deploy "$APP_ENV" "$IMAGE_TAG"
}

main "$@"
```

Argumenti skripte i getopts

```
usage() {
  echo "Usage: $0 --env <staging|production> --tag <image-tag> [--dry-run]"
  exit 1
}

parse_args() {
  DRY_RUN=false

  while [[ $# -gt 0 ]]; do
    case "$1" in
      --env)    APP_ENV="$2";    shift 2 ;;
      --tag)    IMAGE_TAG="$2";  shift 2 ;;
      --dry-run) DRY_RUN=true;  shift   ;;
      -h|--help) usage ;;
      *) echo "ERROR: nepoznata opcija '$1'" >&2; usage ;;
    esac
  done

  [[ -n "${APP_ENV:-}" ]] || { echo "ERROR: --env je obavezan" >&2; usage; }
  [[ -n "${IMAGE_TAG:-}" ]] || { echo "ERROR: --tag je obavezan" >&2; usage; }
}

main() {
  parse_args "$@"
  echo "Deploying $IMAGE_TAG na $APP_ENV (dry_run=$DRY_RUN)"
}

main "$@"
```

Vjezba

Napiši skriptu `rollout.sh`: - Prima `--env` (obavezno), `--image` (obavezno), `--namespace` (default: `default`), `--dry-run` - Funkcija `validate_env()` — provjeri da je env staging ili production - Funkcija `current_image()` — vrati trenutni image iz kubectl (ili echo “unknown” ako kubectl nije dostupan) - Funkcija `do_rollout()` — u dry-run modu ispiši što bi uradila, inače pozovi `kubectl set image` - U main: validiraj, ispiši trenutni image, uradi rollout, ispiši poruku o uspjehu

Source: `03-shell-scripting-for-ops/04-text-processing-jq-yq.md`

Shell — `04` Text processing, jq i yq

Zasto: 90% ops posla je parsovanje outputa — logovi, JSON iz AWS CLI i kubectl, YAML manifesti. Bez alata za ovo kopiraš i lijepiš ručno, što ne skalira i pravi greške.

grep, sed, awk — za nestrukturirani tekst

```
# grep — pretraga i filtriranje
grep "ERROR" app.log                    # linije koje sadrže ERROR
grep -i "error" app.log                 # case-insensitive
grep -v "DEBUG" app.log                 # linije koje NE sadrže DEBUG
grep -c "ERROR" app.log                 # broj matching linija
grep -n "ERROR" app.log                 # sa brojevima linija
grep -E "ERROR|WARN" app.log            # extended regex — više patterna

# U skriptama: -q za tihu provjeru, koristi exit kod
if grep -q "FATAL" app.log; then
  echo "Nađen FATAL error" >&2
  exit 1
fi

# awk — ekstrakovanje kolona iz formatiranog outputa
# $1, $2... su kolone razdvojene whitespaceom
kubectl get pods | awk '{print $1, $3}'          # ime i status
docker ps | awk 'NR>1 {print $NF}'               # zadnja kolona, preskoči header (NR>1)
awk -F: '{print $1}' /etc/passwd                 # custom delimiter
df -h | awk '$5 > 80 {print "WARN:", $0}'        # linije gdje 5. kolona > 80

# sed — zamjena teksta
sed 's/old/new/g' file.txt                       # zamjena sveg
sed -i 's/v1.0.0/v1.1.0/g' deployment.yaml      # in-place edit
sed '/^#/d' config.txt                           # brisanje linija koje počinju sa #
sed -n '10,20p' large.log                        # ispiši linije 10-20
```

Kada NE koristiti sed/awk: za JSON i YAML uvijek koristi jq/yq. `sed` na JSON-u pukne čim se promijeni formatiranje.

---

jq — JSON je svugdje u ops

Svaki `aws` CLI poziv, `kubectl`, `docker inspect`, `curl` na API vraća JSON.

```
# Osnove — čitanje
aws ec2 describe-instances | jq '.Reservations[0].Instances[0].InstanceId'
# Output: "i-1234567890abcdef0"

# -r (raw) uklanja navodnike — potrebno kad rezultat koristiš u skripti
INSTANCE_ID=$(aws ec2 describe-instances | jq -r '.Reservations[0].Instances[0].InstanceId')

# Iteracija kroz listu
kubectl get pods -o json | jq -r '.items[].metadata.name'
# Output:
# myapp-abc123
# myapp-def456

# select() — filtriranje
kubectl get pods -o json | jq -r '.items[] | select(.status.phase == "Running") | .metadata.name'

# Ekstrakovanje više polja odjednom
kubectl get pods -o json | jq -r '.items[] | "\(.metadata.name) \(.status.phase)"'
# Output:
# myapp-abc123 Running
# myapp-def456 Pending

# Realni primjer — uzmi private IP svih running EC2 sa tagom Environment=staging
aws ec2 describe-instances \
  --filters "Name=tag:Environment,Values=staging" "Name=instance-state-name,Values=running" \
  | jq -r '.Reservations[].Instances[].PrivateIpAddress'

# -e — izlazi s greškom ako je rezultat null (korisno u skriptama)
IMAGE=$(kubectl get deployment myapp -o json | jq -re '.spec.template.spec.containers[0].image')
```

---

yq — YAML manifesti u skriptama

`yq` (mikefarah/yq — provjeri verziju, postoje dva nespojiva alata s istim imenom).

```
# Čitanje
yq '.image.tag' values.yaml
yq '.spec.replicas' deployment.yaml
yq '.services.app.image' docker-compose.yml

# Pisanje in-place
yq -i '.image.tag = "v1.2.3"' values.yaml
yq -i '.spec.replicas = 3' deployment.yaml

# U CI — bumping image taga prije helma
NEW_TAG="${CI_COMMIT_SHA:0:8}"
yq -i ".image.tag = \"$NEW_TAG\"" helm/values-staging.yaml
git add helm/values-staging.yaml
git commit -m "ci: bump image tag to $NEW_TAG"

# Čitanje u varijablu
CURRENT_TAG=$(yq '.image.tag' values.yaml)
echo "Trenutni tag: $CURRENT_TAG"

# YAML → JSON bridge za jq (kad trebas kompleksne upite)
yq -o=json deployment.yaml | jq '.spec.template.spec.containers[0].resources'
```

Pipeline kompozicija — spajanje alata

```
# Nađi sve 5xx greške u access logu u posljednjem satu, grupiši po IP-u
grep "$(date -d '1 hour ago' '+%d/%b/%Y:%H')" /var/log/nginx/access.log \
  | awk '$9 ~ /^5/ {print $1}' \
  | sort | uniq -c | sort -rn \
  | head -20

# Provjeri disk usage na svim K8s nodovima
kubectl get nodes -o json \
  | jq -r '.items[].metadata.name' \
  | while IFS= read -r node; do
      echo -n "$node: "
      kubectl describe node "$node" | grep "ephemeral-storage" | tail -1
    done

# Uzmi sve ECS taskove koji su stali s greškom i ispiši razlog
aws ecs list-tasks --cluster prod --desired-status STOPPED \
  | jq -r '.taskArns[]' \
  | xargs -I {} aws ecs describe-tasks --cluster prod --tasks {} \
  | jq -r '.tasks[] | "\(.taskArn | split("/")[-1]): \(.stoppedReason)"'
```

Vjezba

Napiši skriptu `pod-report.sh`: - Uzima namespace kao argument (default: `default`) - Koristi `kubectl get pods -o json` i `jq` da ispiše tabelu: `IME | STATUS | RESTARTS | IMAGE` - Sortira po broju restarta (najvise na vrhu) koristeći `sort` - Na kraju ispiše broj podova koji su u stanju != Running - Ako je broj takvih podova > 0, izlazi sa kodom 1

Source: `03-shell-scripting-for-ops/05-error-handling-trap-and-retry.md`

Shell — `05` Error handling, trap i retry

Zasto: Skripta koja pukne na pola deploya i ostavi klaster u nekonzistentnom stanju je gora od skripte koja nikad nije ni pokrenuta. `trap` i retry logika su razlika između skripta koja se može koristiti u produkciji i one koja je demo.

---

trap — cleanup koji se uvijek izvrši

`trap` registruje funkciju koja se poziva kad skripta završi — normalno, na grešci, ili na signal.

```
#!/usr/bin/env bash
set -euo pipefail

TEMP_DIR=""
DEPLOYMENT_STARTED=false

cleanup() {
  local exit_code=$?

  # Uvijek briši temp fajlove
  [[ -n "$TEMP_DIR" && -d "$TEMP_DIR" ]] && rm -rf "$TEMP_DIR"

  # Rollback samo ako je deploy počeo i nije uspio
  if [[ "$DEPLOYMENT_STARTED" == "true" && $exit_code -ne 0 ]]; then
    echo "Deploy pukao, pokrećem rollback..." >&2
    kubectl rollout undo deployment/myapp
  fi

  exit $exit_code
}

# Registruj cleanup za svaki izlaz
trap cleanup EXIT

# Registruj ERR da vidiš gdje je puklo
trap 'echo "ERROR na liniji $LINENO: $BASH_COMMAND" >&2' ERR

main() {
  TEMP_DIR=$(mktemp -d)

  # Radi nešto s temp direktorijem
  cp deployment.yaml "$TEMP_DIR/"
  yq -i '.image.tag = "'"$IMAGE_TAG"'"' "$TEMP_DIR/deployment.yaml"

  DEPLOYMENT_STARTED=true
  kubectl apply -f "$TEMP_DIR/deployment.yaml"
  kubectl rollout status deployment/myapp --timeout=120s
}

main "$@"
```

Retry logika za transient greške

Mreža je nestabilna, registry je spor, K8s API server je zauzet. Retry s backoffom je standard.

```
# Osnovna retry funkcija — koristi je za svaku flaky operaciju
retry() {
  local max_attempts="$1"
  local sleep_seconds="$2"
  shift 2
  local cmd=("$@")

  local attempt=1
  while (( attempt <= max_attempts )); do
    echo "Pokušaj $attempt/$max_attempts: ${cmd[*]}" >&2
    if "${cmd[@]}"; then
      return 0
    fi

    if (( attempt < max_attempts )); then
      echo "Neuspjelo, čekam ${sleep_seconds}s..." >&2
      sleep "$sleep_seconds"
      sleep_seconds=$(( sleep_seconds * 2 ))  # exponential backoff
    fi
    (( attempt++ ))
  done

  echo "ERROR: svi pokušaji neuspjeli" >&2
  return 1
}

# Upotreba
retry 3 5 docker pull "registry.example.com/myapp:$IMAGE_TAG"
retry 5 10 curl --fail --silent "$HEALTH_URL"
```

Health check loop — čekaj dok servis ne bude spreman

```
wait_for_healthy() {
  local url="$1"
  local timeout="${2:-120}"
  local interval=5
  local elapsed=0

  echo "Čekam da $url bude zdrav..."

  while (( elapsed < timeout )); do
    if curl --fail --silent --max-time 3 "$url" &>/dev/null; then
      echo "Servis je zdrav nakon ${elapsed}s"
      return 0
    fi

    sleep "$interval"
    elapsed=$(( elapsed + interval ))
    echo "  ...još čekam (${elapsed}s/${timeout}s)"
  done

  echo "ERROR: servis nije podignut za ${timeout}s" >&2
  return 1
}

# Upotreba
wait_for_healthy "https://staging.example.com/health" 180
```

Lockfile — spriječi paralelne pokretaje

```
readonly LOCK_FILE="/tmp/${SCRIPT_NAME}.lock"

acquire_lock() {
  if ! mkdir "$LOCK_FILE" 2>/dev/null; then
    local pid
    pid=$(cat "$LOCK_FILE/pid" 2>/dev/null || echo "nepoznat")
    echo "ERROR: skripta već radi (PID: $pid)" >&2
    exit 1
  fi
  echo $$ > "$LOCK_FILE/pid"
}

release_lock() {
  rm -rf "$LOCK_FILE"
}

# U cleanup funkciji uvijek oslobodi lock
cleanup() {
  release_lock
  # ...ostali cleanup
}

trap cleanup EXIT
acquire_lock
```

Vjezba

Napiši skriptu `deploy-with-recovery.sh`: - Pravi temp direktorij na početku, briše ga na kraju (uvijek, i na grešci) - Kopira `deployment.yaml` u temp dir, updateuje image tag - `DEPLOYMENT_STARTED` flag — postavi na `true` tek kad pozoveš kubectl apply - Ako kubectl apply ili rollout status pukne: automatski uradi `kubectl rollout undo` - Koristi `wait_for_healthy` da provjeri `$HEALTH_URL` nakon deploya (3 pokušaja, 30s timeout) - Na svakom koraku jasna poruka šta se dešava

---

Source: `03-shell-scripting-for-ops/06-idempotency-and-cron-environments.md`

Shell — `06` Idempotentnost i cron okruženje

Zasto: Ops skripte se pokreću više puta — iz CI, iz crona, ručno tokom incidenta. Skripta koja pukne na drugom pokretanju, ili koja radi lokalno ali ne u cronu, je skoro beskorisna u produkciji.

Idempotentnost — check before act

Idempotentna skripta daje isti rezultat bez obzira koliko puta je pokreneš.

```
# NIJE idempotentno — dodaje duplikate svaki put
echo "192.168.1.10 db.internal" >> /etc/hosts

# JEST idempotentno — dodaje samo ako ne postoji
if ! grep -qF "db.internal" /etc/hosts; then
  echo "192.168.1.10 db.internal" >> /etc/hosts
fi

# mkdir — uvijek koristiti -p, tiho ako postoji
mkdir -p /opt/app/config /opt/app/logs /opt/app/data

# Instalacija paketa — idempotentno
if ! command -v nginx &>/dev/null; then
  apt-get install -y nginx
fi
# Ili jednostavnije (apt je idempotentan):
apt-get install -y nginx   # ne radi ništa ako je već instaliran

# Kreiranje usera
if ! id "appuser" &>/dev/null; then
  useradd --system --no-create-home appuser
fi

# Kopiranje config fajla — samo ako se promijenio
if ! cmp -s /tmp/nginx.conf /etc/nginx/nginx.conf; then
  cp /tmp/nginx.conf /etc/nginx/nginx.conf
  systemctl reload nginx
fi
```

Cron okruženje — zašto “radi lokalno, ne u cronu”

Cron startuje minimalnu ljusku. Nema tvoj `.bashrc`, `.bash_profile`, `nvm`, `pyenv`, ni ništa što si dodao u PATH.

```
# Cron PATH je samo:
/usr/bin:/bin

# Tvoj interaktivni PATH može biti:
/home/user/.nvm/versions/node/v18/bin:/usr/local/bin:/usr/bin:/bin:...
```

Rješenje 1: Postavi PATH na vrhu skripte

```
#!/usr/bin/env bash
set -euo pipefail

# Eksplicitno postavi PATH koji trebas
export PATH="/usr/local/bin:/usr/bin:/bin"

# Za docker, kubectl, helm koji su u /usr/local/bin:
export PATH="/usr/local/bin:$PATH"
```

Rješenje 2: Koristi apsolutne putanje za kritične komande

```
readonly DOCKER="/usr/bin/docker"
readonly KUBECTL="/usr/local/bin/kubectl"
readonly JQ="/usr/bin/jq"

$KUBECTL get pods    # umjesto: kubectl get pods
```

Logovanje iz crona

Cron obično šalje output na email (koji niko ne čita). Uvijek preusmjeri na fajl:

```
# U crontabu:
0 2 * * * /opt/scripts/cleanup.sh >> /var/log/cleanup.log 2>&1

# Ili sa timestamp u logu:
0 2 * * * /opt/scripts/cleanup.sh >> /var/log/cleanup.log 2>&1; echo "Exit: $?" >> /var/log/
cleanup.log
```

Unutar skripte, dodaj timestamp svim porukama:

```
log() {
  echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"
}

log "Počinjem cleanup..."
log "Obrisano $count fajlova"
```

---

systemd timer — moderni cron

Systemd timeri su bolji od crona jer: loguju u journald, podržavaju `Persistent=true` za propuštene pokretaje, i lakše se debuguju.

```
# /etc/systemd/system/cleanup.timer
[Unit]
Description=Dnevni cleanup logova

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true          # pokreni ako je propušten (server bio ugašen)

[Install]
WantedBy=timers.target
```

```
# /etc/systemd/system/cleanup.service
[Unit]
Description=Log cleanup

[Service]
Type=oneshot
ExecStart=/opt/scripts/cleanup.sh
User=appuser
```

```
systemctl enable --now cleanup.timer
systemctl list-timers          # vidi status
journalctl -u cleanup.service  # vidi logove
```

Debugging “radi lokalno, ne u CI/cronu”

```
# Simuliraj čisto okruženje da reproduciraš problem
env -i HOME=/root PATH=/usr/bin:/bin /bin/bash /opt/scripts/myscript.sh

# Ili u CI doda set -x da vidiš svaku komandu
bash -x /opt/scripts/myscript.sh 2>&1 | head -50
```

Vjezba

Napiši skriptu `log-cleanup.sh` za cron: - Briše log fajlove starije od `RETENTION_DAYS` (default: 7) u `LOG_DIR` (default: `/var/log/app`) - Svaka provjera i brisanje je idempotentno — pokreni dva puta, drugi put ne radi ništa - Na početku eksplicitno

postavlja PATH - Loguje sa timestampom: broj nađenih fajlova, broj obrisanih, ukupno oslobođen prostor - Izlazi s kodom 0 uvijek (greška na jednom fajlu ne smije ubiti cijeli cleanup) - Crontab linija koja ga pokreće u 3:00 svake noći i loguje output

---

Source: `03-shell-scripting-for-ops/07-shellcheck-and-script-quality.md`

Shell — `07` ShellCheck i kvalitet skripti

Zasto: ShellCheck je statički analizator koji hvata klasu grešaka prije nego ih uopće pokreneš. Na poslu, svaka skripta koja ide u CI ili na server mora proći shellcheck — isto kao što kod mora proći linter.

ShellCheck — instaliraj i pokreni

```
# Instalacija
apt-get install shellcheck        # Debian/Ubuntu
brew install shellcheck           # Mac

# Pokretanje
shellcheck deploy.sh

# U CI (GitLab)
shellcheck_job:
  script:
    - shellcheck scripts/*.sh
```

Najčešće greške koje ShellCheck hvata

SC2086 — unquoted variable

```
# ShellCheck: SC2086: Double quote to prevent globbing and word splitting
rm $file           # GREŠKA
rm "$file"         # OK
```

SC2046 — command substitution bez navodnika

```
# ShellCheck: SC2046
for f in $(ls *.log); do    # GREŠKA — ls output ima probleme s razmacima
for f in *.log; do           # OK — direktni glob
```

SC2006 — stari backtick syntax

```
result=`command`     # ShellCheck preporučuje $(command)
result=$(command)    # OK
```

SC2164 — cd bez provjere

```
cd /some/path        # GREŠKA — šta ako dir ne postoji?
cd /some/path || exit 1   # OK
# Ili još bolje:
cd /some/path || { echo "Ne mogu ući u /some/path" >&2; exit 1; }
```

SC2181 — provjera $? direktno

```
command
if [ $? -eq 0 ]; then    # GREŠKA — antipattern

if command; then          # OK — direktna provjera
```

Čitki stil koji olakšava review

```
#!/usr/bin/env bash
set -euo pipefail

# Konstante UPPERCASE
readonly MAX_RETRIES=3
readonly DEPLOY_TIMEOUT=120

# Lokalne varijable lowercase
deploy() {
  local env="$1"
  local tag="$2"
  local namespace="${3:-default}"

  # Duge komande — lom linija sa \
  kubectl set image deployment/myapp \
    "app=registry.example.com/myapp:${tag}" \
    --namespace "$namespace"
}

# Jedna funkcija — jedna odgovornost
verify_deploy() {
  local namespace="$1"
  kubectl rollout status deployment/myapp \
    --namespace "$namespace" \
    --timeout "${DEPLOY_TIMEOUT}s"
}

main() {
  : "${APP_ENV:?}"
  : "${IMAGE_TAG:?}"

  deploy "$APP_ENV" "$IMAGE_TAG"
  verify_deploy "$APP_ENV"
  echo "Deploy uspješan: $IMAGE_TAG na $APP_ENV"
}

main "$@"
```

Pre-commit hook za shellcheck

```
# .git/hooks/pre-commit
#!/usr/bin/env bash
set -euo pipefail

changed_scripts=$(git diff --cached --name-only --diff-filter=ACM | grep '\.sh$' || true)

if [[ -n "$changed_scripts" ]]; then
  echo "Pokrećem shellcheck..."
  # shellcheck disable=SC2086
  shellcheck $changed_scripts
fi
```

```
chmod +x .git/hooks/pre-commit
```

Vjezba

Uzmi bilo koju skriptu iz prethodnih lekcija i: 1. Pokreni `shellcheck` — zabilježi sve warningove 2. Popravi svaki warning — razumij zašto je to bila greška 3. Dodaj pre-commit hook koji blokira commit ako shellcheck ne prođe 4. Provjeri da skripte iz lekcija 01-06 sve prolaze shellcheck bez ikakvih warningova

---

Source: `03-shell-scripting-for-ops/08-integration-project-deploy-pipeline.md`

Shell — `08` Integration project: deploy pipeline skripta

Zasto: Sve prethodne lekcije imaju smisla tek kad ih vidiš zajedno u jednoj pravoj skripti. Ovo je skripta kakvu ćeš pisati i održavati na poslu.

Šta skripta radi

`deploy.sh` — deploy aplikacije na K8s cluster: 1. Validira argumente i env 2. Provjerava da su svi alati prisutni 3. Updateuje image tag u manifestu (yq) 4. Primjenjuje manifest (kubectl apply) 5. Čeka da rollout završi 6. Provjeri zdravlje aplikacije (curl) 7. Na grešci — automatski rollback

---

Kompletna skripta

```bash
#!/usr/bin/env bash
set -euo pipefail

readonly SCRIPT_NAME="$(basename "$0")"
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

# Konfiguracija
readonly HEALTH_CHECK_RETRIES=5
readonly HEALTH_CHECK_INTERVAL=10
readonly ROLLOUT_TIMEOUT=300

# State za cleanup
TEMP_DIR=""
DEPLOYMENT_APPLIED=false

# ─── Logging ──────────────────────────────────────────────────────────────

log()  { echo "[$(date '+%H:%M:%S')] INFO  $*"; }
warn() { echo "[$(date '+%H:%M:%S')] WARN  $*" >&2; }
err()  { echo "[$(date '+%H:%M:%S')] ERROR $*" >&2; }

# ─── Cleanup ──────────────────────────────────────────────────────────────

cleanup() {
  local exit_code=$?

  [[ -n "$TEMP_DIR" && -d "$TEMP_DIR" ]] && rm -rf "$TEMP_DIR"

  if [[ "$DEPLOYMENT_APPLIED" == "true" && $exit_code -ne 0 ]]; then
    warn "Deploy pukao (exit $exit_code), pokrećem rollback..."
    kubectl rollout undo deployment/"$DEPLOYMENT" \
      --namespace "$NAMESPACE" || err "Rollback neuspješan!"
  fi

  exit $exit_code
}

trap cleanup EXIT
trap 'err "Puklo na liniji $LINENO: $BASH_COMMAND"' ERR

# ─── Helpers ──────────────────────────────────────────────────────────────

usage() {
  cat >&2 <<EOF
Usage: $SCRIPT_NAME --env <env> --tag <tag> [opcije]

Obavezno:
  --env <staging|production>   Target okruženje
  --tag <image-tag>            Docker image tag za deploy

Opcionalno:
  --deployment <name>          Naziv K8s deploymenta (default: myapp)
  --namespace <ns>             K8s namespace (default: default)
  --manifest <path>            Path do deployment.yaml (default: deploy/deployment.yaml)
  --health-url <url>           URL za health check (default: bez provjere)
  --dry-run                    Ispiši akcije bez izvršavanja
  -h, --help                   Ova poruka
EOF
  exit 1
}

check_prereqs() {
  local missing=0
  for cmd in kubectl yq jq curl; do
    if ! command -v "$cmd" &>/dev/null; then
      err "Nedostaje alat: $cmd"
```

```
      missing=1
    fi
  done
  (( missing == 0 )) || exit 2
}

parse_args() {
  APP_ENV=""
  IMAGE_TAG=""
  DEPLOYMENT="myapp"
  NAMESPACE="default"
  MANIFEST="${SCRIPT_DIR}/../deploy/deployment.yaml"
  HEALTH_URL=""
  DRY_RUN=false

  while [[ $# -gt 0 ]]; do
    case "$1" in
      --env)        APP_ENV="$2";     shift 2 ;;
      --tag)        IMAGE_TAG="$2";   shift 2 ;;
      --deployment) DEPLOYMENT="$2";  shift 2 ;;
      --namespace)  NAMESPACE="$2";   shift 2 ;;
      --manifest)   MANIFEST="$2";    shift 2 ;;
      --health-url) HEALTH_URL="$2";  shift 2 ;;
      --dry-run)    DRY_RUN=true;     shift   ;;
      -h|--help)    usage ;;
      *) err "Nepoznata opcija: $1"; usage ;;
    esac
  done

  [[ -n "$APP_ENV" ]]   || { err "--env je obavezan";   usage; }
  [[ -n "$IMAGE_TAG" ]] || { err "--tag je obavezan";   usage; }
  [[ "$APP_ENV" =~ ^(staging|production)$ ]] || { err "env mora biti staging ili production"; exit
2; }
  [[ -f "$MANIFEST" ]]  || { err "Manifest ne postoji: $MANIFEST"; exit 2; }
}

# ─── Deploy koraci ──────────────────────────────────────────────────────

prepare_manifest() {
  TEMP_DIR=$(mktemp -d)
  local tmp_manifest="$TEMP_DIR/deployment.yaml"

  cp "$MANIFEST" "$tmp_manifest"

  local current_tag
  current_tag=$(yq '.spec.template.spec.containers[0].image' "$tmp_manifest" | cut -d: -f2)
  log "Trenutni tag: $current_tag → novi tag: $IMAGE_TAG"

  yq -i ".spec.template.spec.containers[0].image |= sub(\":.*\", \":$IMAGE_TAG\")" "$tmp_manifest"
  echo "$tmp_manifest"
}

apply_manifest() {
  local manifest="$1"

  if [[ "$DRY_RUN" == "true" ]]; then
    log "DRY-RUN: kubectl apply -f $manifest"
    kubectl apply -f "$manifest" --dry-run=client
    return
  fi

  kubectl apply -f "$manifest"
  DEPLOYMENT_APPLIED=true
}

wait_for_rollout() {
  if [[ "$DRY_RUN" == "true" ]]; then
```

```
    log "DRY-RUN: kubectl rollout status deployment/$DEPLOYMENT"
    return
  fi

  log "Čekam rollout (max ${ROLLOUT_TIMEOUT}s)..."
  kubectl rollout status deployment/"$DEPLOYMENT" \
    --namespace "$NAMESPACE" \
    --timeout "${ROLLOUT_TIMEOUT}s"
}

verify_health() {
  [[ -n "$HEALTH_URL" ]] || return 0

  local attempt=1
  while (( attempt <= HEALTH_CHECK_RETRIES )); do
    log "Health check $attempt/$HEALTH_CHECK_RETRIES: $HEALTH_URL"

    if curl --fail --silent --max-time 5 "$HEALTH_URL" &>/dev/null; then
      log "Health check OK"
      return 0
    fi

    (( attempt++ ))
    sleep "$HEALTH_CHECK_INTERVAL"
  done

  err "Health check neuspješan nakon $HEALTH_CHECK_RETRIES pokušaja"
  return 1
}

# ─── Main ────────────────────────────────────────────────────────────────

main() {
  parse_args "$@"
  check_prereqs

  log "Deploy: $IMAGE_TAG → $APP_ENV/$NAMESPACE/$DEPLOYMENT"
  [[ "$DRY_RUN" == "true" ]] && warn "DRY-RUN mod — ništa se neće promijeniti"

  local manifest
  manifest=$(prepare_manifest)

  apply_manifest "$manifest"
  wait_for_rollout
  verify_health

  log "✓ Deploy uspješan: $IMAGE_TAG na $APP_ENV"
}

main "$@"
```

Vjezba

1. Pokreni `shellcheck deploy.sh` — popravi sve što ShellCheck nađe
2. Testiraj `--dry-run` mod — provjeri da se ništa ne mijenja
3. Dodaj `--slack-webhook` opciju koja šalje notifikaciju na Slack (curl POST) na kraj deploya i na rollback
4. Dodaj lockfile koji sprečava paralelne deploye na isti env/namespace
5. Napiši GitLab CI job koji poziva ovu skriptu

Phase — 04-python-for-ops

Directory: `04-python-for-ops/`

Source: `04-python-for-ops/01-setup-skeleton-and-cli.md`

Python for ops — `01` Setup, skeleton i CLI

Zasto: Python za ops se ne piše kao aplikacija — nema frejmvorka, nema klasa za sve, nema 10 fajlova. Pišeš skriptu koja ima jasan ulazni tačku, čita argumente, radi stvar, vraća exit kod. Ovo je šablon koji koristiš za svaki ops script.

Setup: venv i requirements

```
# Napravi projekat
mkdir ops-scripts && cd ops-scripts
python3 -m venv .venv
source .venv/bin/activate

# requirements.txt — pin verzije za reproducibilnost
cat > requirements.txt <<EOF
boto3==1.34.0
requests==2.31.0
pyyaml==6.0.1
click==8.1.7
kubernetes==28.1.0
python-json-logger==2.0.7
EOF

pip install -r requirements.txt
```

U CI:

```
# .gitlab-ci.yml
deploy:
  image: python:3.11-slim
  before_script:
    - pip install -r requirements.txt
  script:
    - python scripts/deploy.py --env staging --tag $CI_COMMIT_SHA
```

Minimalni šablon za ops skriptu

```
#!/usr/bin/env python3
"""
deploy.py — deploys application to Kubernetes
"""
import sys
import logging

import click


# ─── Logging ──────────────────────────────────────────────────────────────

def setup_logging(verbose: bool = False) -> None:
    level = logging.DEBUG if verbose else logging.INFO
    logging.basicConfig(
        level=level,
        format="%(asctime)s %(levelname)-8s %(message)s",
        datefmt="%H:%M:%S",
        stream=sys.stderr,  # stderr — ne zagađuj stdout
    )

log = logging.getLogger(__name__)


# ─── CLI ──────────────────────────────────────────────────────────────────

@click.group()
@click.option("--verbose", is_flag=True, help="Debug output")
def cli(verbose: bool) -> None:
    setup_logging(verbose)


@cli.command()
@click.option("--env",   required=True, type=click.Choice(["staging", "production"]))
@click.option("--tag",   required=True, help="Docker image tag")
@click.option("--dry-run", is_flag=True, help="Print actions, don't execute")
def deploy(env: str, tag: str, dry_run: bool) -> None:
    """Deploy application to Kubernetes."""
    log.info("Deploying %s to %s (dry_run=%s)", tag, env, dry_run)

    if dry_run:
        log.info("DRY-RUN: would apply deployment with tag %s", tag)
        return

    # ... logika
    log.info("Deploy successful")


@cli.command()
@click.option("--env", required=True, type=click.Choice(["staging", "production"]))
def rollback(env: str) -> None:
    """Rollback to previous deployment."""
    log.info("Rolling back %s", env)
    # ... logika


# ─── Entry point ──────────────────────────────────────────────────────────

if __name__ == "__main__":
    cli()
```

```
# Pokretanje
python deploy.py deploy --env staging --tag v1.2.3
python deploy.py deploy --env staging --tag v1.2.3 --dry-run
python deploy.py --help
python deploy.py deploy --help
```

Click osnove — sve što trebaš za ops

```
# Obavezna opcija s validacijom
@click.option("--env", required=True, type=click.Choice(["staging", "production"]))

# Opcija s defaultom
@click.option("--namespace", default="default", show_default=True)

# Flag (boolean)
@click.option("--dry-run", is_flag=True)

# Opcija koja čita iz env varijable ako nije proslijeđena
@click.option("--token", envvar="API_TOKEN", help="API token (ili API_TOKEN env var)")

# Obavezni argument (positional)
@click.argument("filename")

# Više vrijednosti
@click.option("--tag", multiple=True)  # --tag v1 --tag v2
```

Exit kodovi iz Clicka

```
@cli.command()
def deploy(...):
    try:
        do_deploy()
    except DeployError as e:
        log.error("Deploy failed: %s", e)
        sys.exit(1)
    except ConfigError as e:
        log.error("Config error: %s", e)
        sys.exit(2)
```

CI čita exit kod — `sys.exit(1)` označava job kao neuspješan.

Vjezba

Napiši `service-check.py` sa Click CLI-em: - Komanda `check` — prima `--url` (više puta) i `--timeout` (default: 5s) - Za svaki URL: HTTP GET, ispiši `[OK] url` ili `[FAIL] url (status 503)` - Komanda `report` — čita listu URLova iz YAML fajla (`urls: [...]`) i radi isti check - Izlazi s kodom 1 ako ijedan URL ne odgovara - Logging na stderr, `[OK]/[FAIL]` linije na stdout

---

Source: `04-python-for-ops/02-files-env-subprocess-and-data.md`

Python for ops — `02` Fajlovi, env, subprocess i data

Zasto: Svaka ops skripta čita konfiguraciju, parsuje JSON/YAML iz alata, i poziva shell komande. Ovo su osnove koje koristiš u svakom scriptu — naučiti ih jednom i koristiti svugdje.

Env varijable

```
import os
import sys

# Čitanje — nikad nemoj hardkodovati credentialse
db_host = os.environ["DB_HOST"]                         # Exception ako nije postavljeno
db_port = int(os.environ.get("DB_PORT", "5432"))        # Default vrijednost
api_token = os.environ.get("API_TOKEN")                 # None ako nije postavljeno

# Validacija na početku skripte
REQUIRED_ENV = ["DB_HOST", "APP_ENV", "IMAGE_TAG"]

def validate_env() -> None:
    missing = [var for var in REQUIRED_ENV if not os.environ.get(var)]
    if missing:
        print(f"ERROR: Nedostaju env varijable: {', '.join(missing)}", file=sys.stderr)
        sys.exit(2)
```

Fajlovi sa pathlib

```
from pathlib import Path

# Osnove
p = Path("/opt/app/config")
p.exists()              # True/False
p.is_file()             # True/False
p.is_dir()              # True/False

# Čitanje i pisanje
content = Path("config.yaml").read_text()
Path("output.json").write_text('{"status": "ok"}')

# mkdir
Path("/opt/app/logs").mkdir(parents=True, exist_ok=True)  # uvijek exist_ok=True

# Glob
for log_file in Path("/var/log/app").glob("*.log"):
    print(log_file.name, log_file.stat().st_size)

# Relativno od skripte
SCRIPT_DIR = Path(__file__).parent
MANIFEST = SCRIPT_DIR / ".." / "deploy" / "deployment.yaml"
```

JSON i YAML

```
import json
import yaml  # pip install pyyaml

# JSON — iz string ili fajla
data = json.loads('{"status": "running", "replicas": 3}')
data = json.load(open("response.json"))
print(json.dumps(data, indent=2))

# Realni primjer — parsovanje kubectl outputa
import subprocess
result = subprocess.run(
    ["kubectl", "get", "deployment", "myapp", "-o", "json"],
    capture_output=True, text=True, check=True
)
deployment = json.loads(result.stdout)
image = deployment["spec"]["template"]["spec"]["containers"][0]["image"]
replicas = deployment["status"]["readyReplicas"]
print(f"Image: {image}, Ready: {replicas}")

# YAML — uvijek safe_load, nikad load (sigurnosni rizik)
with open("values.yaml") as f:
    values = yaml.safe_load(f)

values["image"]["tag"] = "v1.2.3"

with open("values.yaml", "w") as f:
    yaml.dump(values, f, default_flow_style=False)
```

subprocess — pokretanje komandi

```
import subprocess

# Osnova — capture_output=True da uhvatiš stdout/stderr
result = subprocess.run(
    ["kubectl", "get", "pods", "-n", "production"],
    capture_output=True,
    text=True,        # decode bytes → str automatski
    check=False       # ne baci exception na non-zero (mi provjeravamo ručno)
)

if result.returncode != 0:
    print(f"ERROR: kubectl failed:\n{result.stderr}", file=sys.stderr)
    sys.exit(1)

print(result.stdout)

# check=True — baci CalledProcessError na non-zero
try:
    result = subprocess.run(
        ["docker", "push", f"registry.example.com/myapp:{tag}"],
        capture_output=True, text=True, check=True
    )
except subprocess.CalledProcessError as e:
    print(f"Docker push failed: {e.stderr}", file=sys.stderr)
    sys.exit(1)

# NIKAD shell=True s korisničkim inputom — injection risk
# GREŠKA:
subprocess.run(f"kubectl delete pod {pod_name}", shell=True)  # injection ako pod_name = "x; rm -rf /"

# ISPRAVNO — lista argumenata
subprocess.run(["kubectl", "delete", "pod", pod_name], check=True)

# Čitanje jedne vrijednosti
sha = subprocess.run(
    ["git", "rev-parse", "--short", "HEAD"],
    capture_output=True, text=True, check=True
).stdout.strip()
```

HTTP sa requests

```
import requests

# GET sa timeoutom (uvijek postavi timeout!)
response = requests.get("https://api.example.com/health", timeout=10)
response.raise_for_status()  # baci exception za 4xx/5xx
data = response.json()

# POST JSON
response = requests.post(
    "https://hooks.slack.com/services/...",
    json={"text": "Deploy uspješan: v1.2.3 na staging"},
    timeout=10
)
response.raise_for_status()

# Auth
response = requests.get(
    "https://api.example.com/v1/pods",
    headers={"Authorization": f"Bearer {api_token}"},
    timeout=10
)

# Health check s retry
def wait_healthy(url: str, retries: int = 5, delay: int = 10) -> bool:
    for attempt in range(1, retries + 1):
        try:
            r = requests.get(url, timeout=5)
            if r.status_code == 200:
                return True
        except requests.RequestException:
            pass

        if attempt < retries:
            import time
            time.sleep(delay)

    return False
```

Vjezba

Napiši `manifest-updater.py` : - Prima kao argumente: path do `deployment.yaml` , novi image tag - Čita YAML, updateuje `spec.template.spec.containers[0].image` tag - Validira da fajl postoji i da ima očekivanu strukturu (baci jasnu grešku ako ne) - Pokrenuti `kubectl apply -f` na updateovanom fajlu putem subprocess - Sačekati da rollout završi: `kubectl rollout status` (timeout 120s) - Logging na stderr, exit 0/1

Source: `04-python-for-ops/03-boto3-aws-automation.md`

Python for ops — `03` boto3: AWS automation

Zasto: Svaka AWS akcija koju radiš kliktanjem u konzoli može biti Python skripta u CI pipelinu. boto3 je zvanični AWS SDK za Python i temelj svakog ops automatizacije na AWS-u.

Auth i session — osnova svega

```
import boto3
import botocore.exceptions

# Kako boto3 traži credentials (prioritet):
# 1. Env varijable: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY
# 2. ~/.aws/credentials (profili)
# 3. IAM instance role (EC2)
# 4. ECS task role
# 5. EKS pod role (IRSA)

# U produkciji NIKAD ne stavljaj ključeve u kod ili env varijable
# Koristiti IAM role (instance/task/pod role)

# Eksplicitna session — preporučeno, izbjegava globalni state
session = boto3.Session(
    region_name="eu-central-1",
    profile_name="staging",  # ~/.aws/credentials profil (samo lokalno)
)

ec2 = session.client("ec2")
s3  = session.client("s3")
ecs = session.client("ecs")

# Assume role — cross-account ili privilegovane operacije
def assume_role(role_arn: str, session_name: str = "ops-script") -> boto3.Session:
    sts = boto3.client("sts")
    creds = sts.assume_role(
        RoleArn=role_arn,
        RoleSessionName=session_name,
    )["Credentials"]

    return boto3.Session(
        aws_access_key_id=creds["AccessKeyId"],
        aws_secret_access_key=creds["SecretAccessKey"],
        aws_session_token=creds["SessionToken"],
    )
```

Error handling — isti pattern za sve servise

```
from botocore.exceptions import ClientError, NoCredentialsError

def safe_call(fn, *args, **kwargs):
    try:
        return fn(*args, **kwargs)
    except NoCredentialsError:
        print("ERROR: AWS credentials nisu konfigurisani", file=sys.stderr)
        sys.exit(2)
    except ClientError as e:
        code = e.response["Error"]["Code"]
        msg  = e.response["Error"]["Message"]
        print(f"ERROR: AWS API error [{code}]: {msg}", file=sys.stderr)
        raise

# Specifična greška — branch po kodu
try:
    s3.head_bucket(Bucket=bucket_name)
except ClientError as e:
    if e.response["Error"]["Code"] == "404":
        print(f"Bucket {bucket_name} ne postoji")
    elif e.response["Error"]["Code"] == "403":
        print(f"Nemaš pristup bucketu {bucket_name}")
    else:
        raise
```

EC2 — najčešće operacije

```
# Lista instanci sa filterima
def list_instances(env: str, session: boto3.Session) -> list[dict]:
    ec2 = session.client("ec2")
    paginator = ec2.get_paginator("describe_instances")

    instances = []
    for page in paginator.paginate(
        Filters=[
            {"Name": "tag:Environment", "Values": [env]},
            {"Name": "instance-state-name", "Values": ["running"]},
        ]
    ):
        for reservation in page["Reservations"]:
            for inst in reservation["Instances"]:
                instances.append({
                    "id":         inst["InstanceId"],
                    "private_ip": inst.get("PrivateIpAddress", ""),
                    "state":      inst["State"]["Name"],
                    "tags":       {t["Key"]: t["Value"] for t in inst.get("Tags", [])},
                })
    return instances

# Stop instanci i čekanje
def stop_instances(instance_ids: list[str], session: boto3.Session) -> None:
    ec2 = session.client("ec2")
    ec2.stop_instances(InstanceIds=instance_ids)

    waiter = ec2.get_waiter("instance_stopped")
    waiter.wait(InstanceIds=instance_ids)
    print(f"Stopovano: {instance_ids}")
```

S3 — backup i artefakti

```
from datetime import datetime

def upload_backup(local_path: str, bucket: str, prefix: str, session: boto3.Session) -> str:
    s3 = session.client("s3")

    timestamp = datetime.utcnow().strftime("%Y/%m/%d")
    filename = Path(local_path).name
    key = f"{prefix}/{timestamp}/{filename}"

    s3.upload_file(
        local_path,
        bucket,
        key,
        ExtraArgs={"ServerSideEncryption": "AES256"},
    )
    print(f"Uploadovano: s3://{bucket}/{key}")
    return key

def cleanup_old_backups(bucket: str, prefix: str, keep_days: int, session: boto3.Session) -> int:
    s3 = session.client("s3")
    paginator = s3.get_paginator("list_objects_v2")

    cutoff = datetime.utcnow().timestamp() - (keep_days * 86400)
    to_delete = []

    for page in paginator.paginate(Bucket=bucket, Prefix=prefix):
        for obj in page.get("Contents", []):
            if obj["LastModified"].timestamp() < cutoff:
                to_delete.append({"Key": obj["Key"]})

    if to_delete:
        # Batch delete, max 1000 po pozivu
        for i in range(0, len(to_delete), 1000):
            s3.delete_objects(Bucket=bucket, Delete={"Objects": to_delete[i:i+1000]})

    return len(to_delete)
```

ECS — deploy i monitoring

```
def force_deploy(cluster: str, service: str, session: boto3.Session) -> None:
    ecs = session.client("ecs")
    ecs.update_service(
        cluster=cluster,
        service=service,
        forceNewDeployment=True,
    )
    print(f"Deploy pokrenut: {cluster}/{service}")

def wait_stable(cluster: str, service: str, session: boto3.Session, timeout: int = 300) -> None:
    import time
    ecs = session.client("ecs")

    deadline = time.time() + timeout
    while time.time() < deadline:
        resp = ecs.describe_services(cluster=cluster, services=[service])
        svc = resp["services"][0]

        running  = svc["runningCount"]
        desired  = svc["desiredCount"]
        deploys  = len(svc["deployments"])

        print(f"  running={running}/{desired}, deployments={deploys}")

        if running == desired and deploys == 1:
            print("Servis je stabilan")
            return

        time.sleep(10)

    raise TimeoutError(f"Servis nije stabilan nakon {timeout}s")

# Secrets Manager
def get_secret(secret_id: str, session: boto3.Session) -> dict:
    sm = session.client("secretsmanager")
    response = sm.get_secret_value(SecretId=secret_id)
    return json.loads(response["SecretString"])
```

Vjezba

Napiši `aws-cleanup.py` sa Click CLI-em: - Komanda `list-stopped` — lista sve stopovane EC2 instance u env-u (tag `Environment`) - Komanda `terminate-old` — terminira stopovane instance starije od `--days` (default: 7), sa `--dry-run` opcionom - Komanda `backup-logs` — uploaduje sve `.log` fajlove iz direktorija na S3, briše lokalne kopije starije od 3 dana - Proper error handling za ClientError, logging, exit kodovi

---

Source: `04-python-for-ops/04-kubernetes-client.md`

Python for ops — `04` Kubernetes Python client

Zasto: `kubectl` u bash skripti je dovoljno za 80% slučajeva. Python K8s client trebaš kad logika postane složena — filtriranje po više kriterija, reagovanje na stanje clustera, custom automation koja kubectl teško izražava.

Setup i konekcija

```
from kubernetes import client, config, watch

# Iz lokalnog kubeconfig (~/.kube/config)
config.load_kube_config()

# Iz in-cluster service accounta (u K8s podu)
# config.load_incluster_config()

# API klijenti — po grupi resursa
v1      = client.CoreV1Api()       # pods, services, configmaps, secrets, nodes
apps_v1 = client.AppsV1Api()       # deployments, statefulsets, daemonsets
batch   = client.BatchV1Api()      # jobs, cronjobs
```

Čitanje stanja clustera

```
# Lista podova
pods = v1.list_namespaced_pod(namespace="production")
for pod in pods.items:
    name    = pod.metadata.name
    phase   = pod.status.phase
    node    = pod.spec.node_name
    ready   = all(c.ready for c in (pod.status.conditions or []) if c.type == "Ready")
    print(f"{name:40} {phase:10} {node}")

# Filter po labelu
pods = v1.list_namespaced_pod(
    namespace="production",
    label_selector="app=myapp,version=v2"
)

# Restartovi i problemi
def find_crashing_pods(namespace: str) -> list[dict]:
    problems = []
    pods = v1.list_namespaced_pod(namespace=namespace)

    for pod in pods.items:
        for container in (pod.status.container_statuses or []):
            if container.restart_count > 5:
                problems.append({
                    "pod":      pod.metadata.name,
                    "container": container.name,
                    "restarts": container.restart_count,
                    "state":    str(container.state),
                })
    return problems

# Logovi
logs = v1.read_namespaced_pod_log(
    name="myapp-abc123",
    namespace="production",
    tail_lines=100,
    container="app",  # ako pod ima više containera
)
print(logs)
```

Update deploymenta

```
def update_image(deployment: str, namespace: str, new_image: str, dry_run: bool = False) -> None:
    """Zamijeni image u prvom containeru deploymenta."""

    # Čitaj trenutno stanje
    dep = apps_v1.read_namespaced_deployment(name=deployment, namespace=namespace)
    old_image = dep.spec.template.spec.containers[0].image

    print(f"{deployment}: {old_image} → {new_image}")

    if dry_run:
        print("DRY-RUN: nije primijenjeno")
        return

    # Patch — samo šalje razliku
    patch = {
        "spec": {
            "template": {
                "spec": {
                    "containers": [{"name": dep.spec.template.spec.containers[0].name, "image": new_im
age}]
                }
            }
        }
    }
    apps_v1.patch_namespaced_deployment(name=deployment, namespace=namespace, body=patch)
    print("Patch primijenjen")


def wait_rollout(deployment: str, namespace: str, timeout: int = 300) -> None:
    """Čekaj da rollout završi — polling na deployment status."""
    import time
    deadline = time.time() + timeout

    while time.time() < deadline:
        dep = apps_v1.read_namespaced_deployment(name=deployment, namespace=namespace)
        desired    = dep.spec.replicas
        updated    = dep.status.updated_replicas or 0
        available  = dep.status.available_replicas or 0

        print(f"  updated={updated}/{desired}, available={available}/{desired}")

        if updated == desired and available == desired:
            print("Rollout završen")
            return

        time.sleep(10)

    raise TimeoutError(f"Rollout nije završen za {timeout}s")
```

Watch — reaguj na promjene u clusteru

```
def watch_pods(namespace: str, label_selector: str) -> None:
    """Stream eventova o podovima — korisno za monitoring i debugging."""
    w = watch.Watch()

    for event in w.stream(
        v1.list_namespaced_pod,
        namespace=namespace,
        label_selector=label_selector,
        timeout_seconds=300,
    ):
        event_type = event["type"]         # ADDED, MODIFIED, DELETED
        pod = event["object"]
        phase = pod.status.phase
        name = pod.metadata.name

        print(f"{event_type}: {name} ({phase})")

        if phase == "Failed":
            print(f"  ALERT: Pod {name} pao!")
            # Pošalji notifikaciju, uzmi logove...
```

Čišćenje starih jobova

```
def cleanup_completed_jobs(namespace: str, dry_run: bool = False) -> int:
    """Briši završene K8s jobove — čest maintenance task."""
    jobs = batch.list_namespaced_job(namespace=namespace)
    deleted = 0

    for job in jobs.items:
        succeeded = job.status.succeeded or 0
        if succeeded > 0:
            print(f"Brišem završeni job: {job.metadata.name}")

            if not dry_run:
                batch.delete_namespaced_job(
                    name=job.metadata.name,
                    namespace=namespace,
                    body=client.V1DeleteOptions(propagation_policy="Foreground"),
                )
            deleted += 1

    return deleted
```

Vjezba

Napiši `cluster-health.py` sa Click CLI-em: - Komanda `pods` — ispiši tabelu: namespace, ime, status, restartovi, node; sortiramo po restartima - Komanda `images` — listu svih unique imaga u clusteru (svi deploymenti, svi namespacei) - Komanda `cleanup-jobs` — briši completed jobove, `--dry-run` podrška - Ako nađeš pod s > 10 restartatova, ispiši zadnjih 50 linija loga i upozorenje

---

Source: `04-python-for-ops/05-error-handling-logging-and-testing.md`

Python for ops — `05` Error handling, logging i testiranje skripti

Zasto: Ops skripta koja pukne s `KeyError: 'InstanceId'` u 2 ujutro je problem. Dobro structurirani error handling znači da greška kaže gdje je nastala, zašto, i šta je skripta pokušala da uradi. Testovi znače da možeš refaktorisati bez straha.

Error handling — specifičan, ne generalan

```
# LOŠE — hvata sve, skriva pravi problem
try:
    do_deploy()
except Exception as e:
    print(f"Greška: {e}")
    sys.exit(1)

# DOBRO — specifične greške, jasne poruke
from botocore.exceptions import ClientError, NoCredentialsError
import subprocess

class DeployError(Exception):
    """Deployment neuspješan."""

class ConfigError(Exception):
    """Pogrešna konfiguracija ili nedostaju env varijable."""

def deploy(env: str, tag: str) -> None:
    try:
        _update_ecs_service(env, tag)
    except ClientError as e:
        code = e.response["Error"]["Code"]
        raise DeployError(f"ECS update neuspješan [{code}]: {e}") from e
    except subprocess.CalledProcessError as e:
        raise DeployError(f"kubectl komanda pala (exit {e.returncode}): {e.stderr}") from e

def main():
    try:
        deploy(env, tag)
    except ConfigError as e:
        log.error("Konfiguracija: %s", e)
        sys.exit(2)
    except DeployError as e:
        log.error("Deploy neuspješan: %s", e)
        sys.exit(1)
    except KeyboardInterrupt:
        log.warning("Prekinuto")
        sys.exit(130)
```

Logging — strukturiran i koristan

```
import logging
import sys
from pythonjsonlogger import jsonlogger  # pip install python-json-logger

def setup_logging(verbose: bool = False) -> None:
    level = logging.DEBUG if verbose else logging.INFO

    handler = logging.StreamHandler(sys.stderr)

    # Produkcija — JSON format koji Loki može parsovati
    if os.environ.get("LOG_FORMAT") == "json":
        formatter = jsonlogger.JsonFormatter(
            "%(asctime)s %(levelname)s %(name)s %(message)s"
        )
    else:
        # Lokalno — čitljiv format
        formatter = logging.Formatter(
            fmt="%(asctime)s %(levelname)-8s %(message)s",
            datefmt="%H:%M:%S",
        )

    handler.setFormatter(formatter)
    logging.basicConfig(level=level, handlers=[handler])

log = logging.getLogger(__name__)

# Upotreba — uvijek s kontekstom, ne samo s porukom
log.info("Pokrećem deploy", extra={"env": env, "tag": tag})
log.warning("Health check neuspješan, pokušavam ponovo", extra={"attempt": 2, "url": url})
log.error("Deploy pao", extra={"error": str(e), "service": service_name})
```

Context manager za cleanup

```
import tempfile
import shutil
from contextlib import contextmanager

@contextmanager
def temp_workspace():
    """Temp direktorij koji se uvijek briše, i na grešci."""
    tmpdir = tempfile.mkdtemp(prefix="ops-deploy-")
    try:
        yield Path(tmpdir)
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True)

# Upotreba
def deploy_with_manifest(tag: str) -> None:
    with temp_workspace() as workspace:
        manifest = workspace / "deployment.yaml"
        shutil.copy("deploy/deployment.yaml", manifest)

        # yq update
        subprocess.run(
            ["yq", "-i", f'.spec.template.spec.containers[0].image |= sub(":.*", ":{tag}")', str(manif
est)],
            check=True
        )

        subprocess.run(["kubectl", "apply", "-f", str(manifest)], check=True)
    # tmpdir je već obrisan čak i ako subprocess pukne
```

Testiranje ops skripti

Ops skripte nisu aplikacije, ali se mogu testirati. Cilj: testirati logiku bez pozivanja AWS/kubectl.

```
# tests/test_deploy.py
import pytest
from unittest.mock import MagicMock, patch

# Testiraj logiku, mock-aj external calls
def test_validate_env_missing():
    with pytest.raises(SystemExit) as exc:
        with patch.dict("os.environ", {}, clear=True):
            validate_env()
    assert exc.value.code == 2

def test_deploy_updates_correct_image():
    mock_ecs = MagicMock()

    with patch("boto3.Session") as mock_session:
        mock_session.return_value.client.return_value = mock_ecs

        deploy_to_ecs("staging", "v1.2.3")

        mock_ecs.update_service.assert_called_once_with(
            cluster="staging-cluster",
            service="myapp",
            forceNewDeployment=True,
        )

def test_wait_stable_succeeds():
    mock_ecs = MagicMock()
    mock_ecs.describe_services.return_value = {
        "services": [{
            "runningCount": 2,
            "desiredCount": 2,
            "deployments": [{"status": "PRIMARY"}],
        }]
    }

    # Ne smije baciti exception
    wait_stable("cluster", "service", mock_ecs, timeout=30)

# Pokretanje
# pytest tests/ -v
```

Vjezba

Uzmi `aws-cleanup.py` iz prethodne lekcije i: 1. Dodaj custom exception klase: `AWSError`, `ConfigError` 2. Refaktoriši error handling — nikad goli `except Exception` 3. Dodaj JSON logging kad je `LOG_FORMAT=json` u env-u 4. Napiši 3 testa za `terminate-old` logiku koristeći mock boto3 — testiraj: dry-run ne terminira, instance mlađe od `--days` se preskačaju, error handling za ClientError

---

Source: `04-python-for-ops/06-integration-project-deploy-tool.md`

Python for ops — `06` Integration project: deploy alat

Zasto: Sve prethodne lekcije su fragmenti. Ovdje ih spajamo u jedan alat koji se zaista koristi na poslu — CLI sa više komandi, boto3, K8s client, health check, notifikacije, testovi.

Šta alat radi

`deploy-tool.py` — kompletni deploy alat: - `deploy` — deploya na ECS Fargate ili K8s (konfiguriše se po env-u) - `rollback` — vraća na prethodnu verziju - `status` — ispiši stanje servisa

Kompletna implementacija

```
#!/usr/bin/env python3
"""deploy-tool.py — Application deployment tool"""

import json
import logging
import os
import subprocess
import sys
import time
from pathlib import Path
from contextlib import contextmanager
from typing import Optional

import boto3
import click
import requests
import yaml
from botocore.exceptions import ClientError, NoCredentialsError
from kubernetes import client as k8s_client, config as k8s_config


# ─── Exceptions ──────────────────────────────────────────────────────────────

class DeployError(Exception): pass
class ConfigError(Exception): pass
class HealthCheckError(Exception): pass


# ─── Logging ─────────────────────────────────────────────────────────────────

def setup_logging(verbose: bool) -> None:
    logging.basicConfig(
        level=logging.DEBUG if verbose else logging.INFO,
        format="%(asctime)s %(levelname)-8s %(message)s",
        datefmt="%H:%M:%S",
        stream=sys.stderr,
    )

log = logging.getLogger(__name__)


# ─── Config ──────────────────────────────────────────────────────────────────

ENV_CONFIG = {
    "staging": {
        "platform":   "ecs",
        "cluster":    "staging-cluster",
        "service":    "myapp-staging",
        "region":     "eu-central-1",
        "health_url": "https://staging.example.com/health",
    },
    "production": {
        "platform":   "k8s",
        "namespace":  "production",
        "deployment": "myapp",
        "health_url": "https://app.example.com/health",
    },
}

def get_config(env: str) -> dict:
    if env not in ENV_CONFIG:
        raise ConfigError(f"Nepoznat env: {env}. Dostupni: {list(ENV_CONFIG)}")
    return ENV_CONFIG[env]


# ─── Health check ────────────────────────────────────────────────────────────
```

```
def wait_healthy(url: str, retries: int = 5, delay: int = 10) -> None:
    for attempt in range(1, retries + 1):
        try:
            r = requests.get(url, timeout=5)
            if r.status_code == 200:
                log.info("Health check OK (%s)", url)
                return
            log.warning("Health check attempt %d/%d: status %d", attempt, retries, r.status_code)
        except requests.RequestException as e:
            log.warning("Health check attempt %d/%d: %s", attempt, retries, e)

        if attempt < retries:
            time.sleep(delay)

    raise HealthCheckError(f"Servis nije zdrav nakon {retries} pokušaja: {url}")


# ─── Slack notifikacija ──────────────────────────────────────────────

def notify(message: str, success: bool = True) -> None:
    webhook = os.environ.get("SLACK_WEBHOOK_URL")
    if not webhook:
        return

    color = "#36a64f" if success else "#d32f2f"
    payload = {
        "attachments": [{
            "color": color,
            "text": message,
            "footer": "deploy-tool",
        }]
    }

    try:
        requests.post(webhook, json=payload, timeout=5)
    except requests.RequestException:
        log.warning("Slack notifikacija neuspješna")


# ─── ECS deploy ──────────────────────────────────────────────────────

def deploy_ecs(tag: str, cfg: dict, dry_run: bool) -> None:
    session = boto3.Session(region_name=cfg["region"])
    ecs = session.client("ecs")

    log.info("ECS deploy: %s/%s → %s", cfg["cluster"], cfg["service"], tag)

    # Uzmi task definition da updateujemo image
    svc = ecs.describe_services(cluster=cfg["cluster"], services=[cfg["service"]])["services"][0]
    task_def_arn = svc["taskDefinition"]
    task_def = ecs.describe_task_definition(taskDefinition=task_def_arn)["taskDefinition"]

    # Update image u task definition
    containers = task_def["containerDefinitions"]
    containers[0]["image"] = f"{containers[0]['image'].rsplit(':', 1)[0]}:{tag}"

    if dry_run:
        log.info("DRY-RUN: would register new task definition and update service")
        return

    # Registruj novu task definition
    new_td = ecs.register_task_definition(
        family=task_def["family"],
        containerDefinitions=containers,
        cpu=task_def["cpu"],
        memory=task_def["memory"],
```

```
        networkMode=task_def["networkMode"],
        requiresCompatibilities=task_def.get("requiresCompatibilities", []),
        executionRoleArn=task_def.get("executionRoleArn"),
        taskRoleArn=task_def.get("taskRoleArn"),
    )["taskDefinition"]["taskDefinitionArn"]

    ecs.update_service(
        cluster=cfg["cluster"],
        service=cfg["service"],
        taskDefinition=new_td,
    )

    # Čekaj stabilan state
    log.info("Čekam stabilan ECS servis...")
    deadline = time.time() + 300
    while time.time() < deadline:
        svc = ecs.describe_services(cluster=cfg["cluster"], services=[cfg["service"]])["services"][0]
        running  = svc["runningCount"]
        desired  = svc["desiredCount"]
        deploys  = len(svc["deployments"])
        log.debug("running=%d/%d, deployments=%d", running, desired, deploys)

        if running == desired and deploys == 1:
            break
        time.sleep(10)
    else:
        raise DeployError("ECS servis nije stabilan nakon 5min")


# ─── K8s deploy ──────────────────────────────────────────────────────

def deploy_k8s(tag: str, cfg: dict, dry_run: bool) -> None:
    k8s_config.load_kube_config()
    apps = k8s_client.AppsV1Api()

    dep = apps.read_namespaced_deployment(
        name=cfg["deployment"],
        namespace=cfg["namespace"]
    )

    old_image = dep.spec.template.spec.containers[0].image
    new_image = f"{old_image.rsplit(':', 1)[0]}:{tag}"

    log.info("K8s deploy: %s → %s", old_image, new_image)

    if dry_run:
        log.info("DRY-RUN: would patch deployment %s", cfg["deployment"])
        return

    patch = {"spec": {"template": {"spec": {"containers": [
        {"name": dep.spec.template.spec.containers[0].name, "image": new_image}
    ]}}}}

    apps.patch_namespaced_deployment(
        name=cfg["deployment"],
        namespace=cfg["namespace"],
        body=patch
    )

    result = subprocess.run(
        ["kubectl", "rollout", "status", f"deployment/{cfg['deployment']}",
         "-n", cfg["namespace"], "--timeout=5m"],
        capture_output=True, text=True
    )
    if result.returncode != 0:
        raise DeployError(f"Rollout neuspješan: {result.stderr}")
```

```
# ─── CLI ──────────────────────────────────────────────────────────────────────

@click.group()
@click.option("--verbose", is_flag=True)
def cli(verbose: bool) -> None:
    setup_logging(verbose)


@cli.command()
@click.option("--env",     required=True, type=click.Choice(["staging", "production"]))
@click.option("--tag",     required=True)
@click.option("--dry-run", is_flag=True)
def deploy(env: str, tag: str, dry_run: bool) -> None:
    """Deploy novu verziju aplikacije."""
    try:
        cfg = get_config(env)

        if cfg["platform"] == "ecs":
            deploy_ecs(tag, cfg, dry_run)
        else:
            deploy_k8s(tag, cfg, dry_run)

        if not dry_run and cfg.get("health_url"):
            wait_healthy(cfg["health_url"])

        msg = f"✓ Deploy uspješan: `{tag}` na `{env}`"
        log.info(msg)
        notify(msg, success=True)

    except (DeployError, HealthCheckError) as e:
        log.error("Deploy neuspješan: %s", e)
        notify(f"✗ Deploy neuspješan na `{env}`: {e}", success=False)
        sys.exit(1)

    except ConfigError as e:
        log.error("Config greška: %s", e)
        sys.exit(2)


@cli.command()
@click.option("--env", required=True, type=click.Choice(["staging", "production"]))
def status(env: str) -> None:
    """Prikaži stanje servisa."""
    cfg = get_config(env)

    if cfg["platform"] == "ecs":
        session = boto3.Session(region_name=cfg["region"])
        ecs = session.client("ecs")
        svc = ecs.describe_services(
            cluster=cfg["cluster"], services=[cfg["service"]]
        )["services"][0]
        print(json.dumps({
            "env":     env,
            "running": svc["runningCount"],
            "desired": svc["desiredCount"],
            "status":  svc["status"],
        }, indent=2))
    else:
        k8s_config.load_kube_config()
        apps = k8s_client.AppsV1Api()
        dep = apps.read_namespaced_deployment(
            name=cfg["deployment"], namespace=cfg["namespace"]
        )
        print(json.dumps({
            "env":       env,
            "available": dep.status.available_replicas,
```

```
            "desired":   dep.spec.replicas,
            "image":     dep.spec.template.spec.containers[0].image,
        }, indent=2))


if __name__ == "__main__":
    cli()
```

Vjezba

1. Pokreni `deploy --env staging --tag v1.0.0 --dry-run` — provjeri da ništa nije pozvano
2. Napiši 4 unit testa: deploy_ecs zove update_service, wait_healthy uspijeva na 3. pokušaju, ConfigError za nepoznat env, rollback poziva rollout undo
3. Dodaj `rollback` komandu koja poziva `kubectl rollout undo` ili ECS rollback na prethodnu task definition
4. Napiši `.gitlab-ci.yml` job koji poziva `deploy --env staging --tag $CI_COMMIT_SHA` i `deploy --env production --tag $CI_COMMIT_TAG` (samo na tagove)

Phase — 05-git-collaboration-and-release-workflow

Directory: `05-git-collaboration-and-release-workflow/`

Source: `05-git-collaboration-and-release-workflow/01-local-git-three-trees-commit-discipline.md`

Unit 01 — Working tree · index · commits

Comfort `git status`, `git diff`, purposeful `git add`, meaningful `git commit`, `git log`, `git restore` choreography.

Laboratory mutate `service.php` (rename sensibly)—inspect diffs stage selectively avoid blind `git add .` habitual damage long-term.

Source: `05-git-collaboration-and-release-workflow/02-branching-and-release-shape.md`

Unit 02 — Parallel development branches

Operate `git branch`, `git switch`; align mental model illustrating `main` ↔protective merges↔experiments.

Author disposable feature branches analogous `feature/login`, `bugfix/token` practising naming clarity organisational standards appreciate.

Source: `05-git-collaboration-and-release-workflow/03-merge-and-conflicts.md`

Unit 03 — Merge outcomes and conflict archaeology

Produce deliberate textual conflicts merging divergent snippets—resolve calmly annotating rationale fast-forward versus merge-commit semantics merit conscious distinction.

Source: `05-git-collaboration-and-release-workflow/04-interactive-rebase-history-shape.md`

Unit 04 — Rebasing responsibly

Squash noisy WIP increments via `git rebase -i` practising commit message uplift—never rewrite published shared history recklessly.

Source: `05-git-collaboration-and-release-workflow/05-cherry-pick-revert-reset-recovery-literacy.md`

Unit 05 — Surgical history manoeuvres

Differentiate cherry-picking isolated fixes versus revert-forward-safe undo paths versus destructive reset boundaries—simulate chaos then recover ethically.

Source: `05-git-collaboration-and-release-workflow/06-tags-semver-release-markers.md`

Unit 06 — Annotated tags and semantic intent

Practice lightweight vs annotated tags signalling release anchors—align verbally with semver philosophy teams adopt—even if branching scheme varies.

Source: `05-git-collaboration-and-release-workflow/07-hooks-quality-guardrails.md`

Unit 07 — Client-side hooks (pre-commit quality)

Author minimal hook scaffolding invoking `phpstan` / unit subset—articulate divergence local developer versus CI redundancy intentionally complementary.

Source: `05-git-collaboration-and-release-workflow/08-reflog-safety-net.md`

Unit 08 — Recover from self-inflicted divergence

Leverage `git reflog` after deleting branch mistakenly or hard resetting—articulate TTL awareness garbage collection nuances high-level—not unlimited safety net myths.

Source: `05-git-collaboration-and-release-workflow/09-end-to-end-team-simulation.md`

Unit 09 — Scripted team rehearsal without lookups

Symfony repo choreography: branching, contentious merges, tagging, hypothetical rollback narration—maintain handwritten timeline proving comprehension separate memorised trivia.

Phase — 06-production-application-network-flow

Directory: `06-production-application-network-flow/`

Source: `06-production-application-network-flow/01-dns-records-and-cidr-literacy.md`

Ops networking traffic unit

dig/nslookup rehearsals plus CIDR addressing vocabulary subnet gateway broadcast clarity.

Source: `06-production-application-network-flow/02-nat-routing-path-traceroute.md`

Ops networking traffic unit

Default gateway traceroute narration articulating asymmetric path misconceptions cautiously ethically.

Source: `06-production-application-network-flow/03-tls-chain-openssl-cli.md`

Ops networking traffic unit

openssl s_client inspection plus nginx TLS misconfiguration breakage recovery lab.

Source: `06-production-application-network-flow/04-nginx-reverse-proxy-upstream-shape.md`

Ops networking traffic unit

Proxy headers upstream selection debugging consciously Symfony upstream analogues.

Source: `06-production-application-network-flow/05-load-balancing-upstream-pool.md`

Ops networking traffic unit

Multi upstream round-robin illustrative combining Go sidecar narrative responsibly.

Source: `06-production-application-network-flow/06-firewall-ufw-thinking.md`

Ops networking traffic unit

Allow/deny rule rehearsal isolating unintended port lockouts consciously recoverable ethically.

Source: `06-production-application-network-flow/07-traefik-dynamic-routes.md`

Ops networking traffic unit

Compose Traefik illustrating automatic router TLS middleware experimentation consciously disposable.

Source: `06-production-application-network-flow/08-cross-layer-troubleshooting-marathon.md`

Ops networking traffic unit

Rotate DNS TLS firewall proxy faults documenting systematic bisection ethically.

Source: `06-production-application-network-flow/09-traefik-nginx-stack-integration.md`

Ops networking traffic unit

Synthetic internet-facing path through proxies to backends plus datastore documenting full trace reasoning.

Phase — 07-docker-containers-image-workflow

Directory: `07-docker-containers-image-workflow/`

Source: `07-docker-containers-image-workflow/01-images-vs-running-containers.md`

Docker — `01 / images / vs / running / containers`

Focus: Pull nginx, run/list/stop/rm lifecycle; articulate immutable image vs ephemeral writable layer intuition.

Source: `07-docker-containers-image-workflow/02-dockerfile-layer-cache-invalidation.md`

Docker — `02 / dockerfile / layer / cache / invalidation`

Focus: Build PHP/FPM-ish sample altering README vs composer lock observing cache churn discipline.

Source: `07-docker-containers-image-workflow/03-entrypoint-vs-cmd.md`

Docker — `03 / entrypoint / vs / cmd`

Focus: Demonstrate interplay default args vs fixed executable rewriting container argv experiments ethically.

Source: `07-docker-containers-image-workflow/04-volumes-and-bind-mount-persistence.md`

Docker — `04 / volumes / and / bind / mount / persistence`

Focus: Postgres ephemeral loss without named volume versus bind-mount dev ergonomics contrasts.

Source: `07-docker-containers-image-workflow/05-bridge-dns-service-resolution.md`

Docker — `05 / bridge / dns / service / resolution`

Focus: Custom network linking Symfony ↔ Redis ↔ Postgres diagnosing miswired DATABASE_URL.

Source: `07-docker-containers-image-workflow/06-docker-compose-multi-service.md`

Docker — `06 / docker / compose / multi / service`

Focus: Compose stack Symfony+Redis+Postgres+Nginx; environment, restart policy, healthcheck posture.

Source: `07-docker-containers-image-workflow/07-multi-stage-build-optimisation.md`

Docker — `07 / multi / stage / build / optimisation`

Focus: Contrast bloated single-stage Symfony builder vs trimmed runtime artefacts measuring image size deltas.

Source: `07-docker-containers-image-workflow/08-registry-tag-push-pull.md`

Docker — `08 / registry / tag / push / pull`

Focus: Tag, push/pull through registry sandbox articulating versioning promotion story.

Source: `07-docker-containers-image-workflow/09-container-debug-tooling.md`

Docker — `09 / container / debug / tooling`

Focus: Logs, inspect, exec, stats while breaking ports/env/volumes/permissions intentionally.

Source: `07-docker-containers-image-workflow/10-compose-integration-blind-rehearsal.md`

Docker — `10 / compose / integration / blind / rehearsal`

Focus: Stand entire stack sans notes degrade recover documenting lifecycle hypotheses.

Source: `07-docker-containers-image-workflow/11-image-hardening-nonroot-capabilities-readonly-rootfs.md`

Unit 11 — Image hardening: non-root, capabilities, read-only roots

As images move toward production-facing clusters, predictable UID/GID, dropped Linux capabilities, and read-only container roots materially reduce escalation blast radius—even before admission controllers enforce profiles.

In labs, rebuild a Symfony or Go image so the main process UID is not `0`, mount writable paths explicitly for cache dirs, and iterate `docker inspect` verifying effective user. Pair with Compose override volume permissions consciously rather than widening host permissions lazily.

Document trade-offs: installers expecting root, PHP-FPM user mapping, ephemeral `/tmp`. Capture one rollback story when hardening silently breaks health checks—that narrative is interview-grade depth.

Source: `07-docker-containers-image-workflow/12-buildkit-secrets-and-deterministic-cache.md`

Unit 12 — BuildKit: build secrets vs cache discipline

BuildKit allows mounting short-lived secrets only during build steps, shrinking the chance they land in immutable layers mirrored to registries. Contrast brittle `COPY` of `.env.build` artefacts—acceptable only on disposable workstations under policy you control explicitly.

Practise: convert one multi-stage Dockerfile to use ephemeral secret mounts fetching private Composer/NPM artefacts; quantify cache invalidation triggers when pinning dependency manifests versus stray README tweaks.

Tie scanners (`18-*`) back in: deterministic rebuilds clarify which layer introduced a CVE escalation when base images churn weekly.

Source: `07-docker-containers-image-workflow/13-image-signing-and-verification-consumer-mindset.md`

Unit 13 — Signing & verification as a consumer

You may not operate the organisational signing KMS today, yet you still must articulate how clusters or CI verifies publisher identity: digest references, Sigstore/Cosign-style flows, immutable tags versus floating `latest` regret stories professionally.

Laboratory analogue: practise referencing images by `@sha256:` in Compose or Helm values on disposable clusters verifying identical deploy reproduces—even if organisational signing infra arrives later ethically.

When verification fails mid-pipeline, write the triage headings you would escalate: artefact mismatch, revoked signature, stale trust root rotation narrative responsibly.

Source: `07-docker-containers-image-workflow/14-registry-governance-retention-supply-chain.md`

Unit 14 — Registry policy, retention, and supply-chain cadence

Professional registries differentiate dev/test promotion lanes, immutable release channels, lifecycle policies pruning ancient layers, robotic vulnerability SLA conversations synchronised with base image rotations practiced earlier.

Exercise: emulate two repositories (`dev` / `rel`) tagging strategy; practise garbage-collection awareness (conceptual—not destroying shared org registries unknowingly ethically). Mirror retention policies legal teams sometimes negotiate around personal data artefacts even when absent here consciously.

Leverage backlog hooks: annotate chart (`09-*`) or pipeline (`04-*`) step promoting only signed digests aligning operational compliance peers expect eventually.

Phase — 08-podman-rootless-runtime-model

Directory: `08-podman-rootless-runtime-model/`

Source: `08-podman-rootless-runtime-model/01-daemonless-runtime-basics.md`

Podman — `01-daemonless-runtime-basics`

Focus: Compare Podman daemonless model vs centralized dockerd—lifecycle parity drills.

Source: `08-podman-rootless-runtime-model/02-rootless-namespaces-privilege-mapping.md`

Podman — `02-rootless-namespaces-privilege-mapping`

Focus: Run workloads rootless illustrating user namespace containment vs host-root conflation myths.

Source: `08-podman-rootless-runtime-model/03-pod-multi-container-shared-net.md`

Podman — `03-pod-multi-container-shared-net`

Focus: Group Symfony+sidecars in pod analogue practising shared network namespace ergonomics prepping Kubernetes mental continuity.

Source: `08-podman-rootless-runtime-model/04-registry-push-pull-podman.md`

Podman — `04-registry-push-pull-podman`

Focus: Image tagging/pushing/pull parity with disciplined auth rotation storytelling.

Source: `08-podman-rootless-runtime-model/05-compose-compat-migration.md`

Podman — `05-compose-compat-migration`

Focus: Migrate earlier compose definitions exercising compatibility caveats responsibly.

Source: `08-podman-rootless-runtime-model/06-podman-network-and-volume-drills.md`

Podman — `06-podman-network-and-volume-drills`

Focus: Break DNS, volume mounts, permission skew—repair methodically documenting.

Source: `08-podman-rootless-runtime-model/07-generate-systemd-unit-for-long-running.md`

Podman — `07-generate-systemd-unit-for-long-running`

Focus: Use `podman generate systemd` enabling restart policies surviving reboot ethically on lab VM.

Source: `08-podman-rootless-runtime-model/08-podman-debug-suite.md`

Podman — `08-podman-debug-suite`

Focus: Logs/inspect/exec fault injection narratives bridging Podman quirks.

Source: `08-podman-rootless-runtime-model/09-rootless-stack-integration-rehearsal.md`

Podman — `09-rootless-stack-integration-rehearsal`

Focus: Full Symfony+deps stack rootless degrade/recover without Docker daemon dependency.

Source: `08-podman-rootless-runtime-model/10-podman-quadlets-systemd-native.md`

Podman — `10-podman-quadlets-systemd-native`

Focus: Replace `podman generate systemd` with Quadlet `.container` unit files — the modern, declarative approach for production-grade rootless containers managed by systemd.

Practise focus

- Write a `.container` Quadlet file in `~/.config/containers/systemd/` and confirm `systemd --user` picks it up
- Map Quadlet fields to their Compose equivalents: `Image`, `Volume`, `Environment`, `Network`, `PublishPort`
- Enable auto-restart and verify container survives reboot without manual intervention
- Use `.network` Quadlet files to wire multi-container stacks without Compose
- Understand why Quadlets supersede `podman generate systemd` — declarative vs generated, drift-safe, reviewable in git
- Debug Quadlet load failures with `systemctl --user status` and `journalctl --user -u`
- Compare: Quadlet vs Compose vs Kubernetes Pod YAML — when each surface fits

---

Phase — 09-container-networking-service-communication

Directory: `09-container-networking-service-communication/`

Source: `09-container-networking-service-communication/01-bridge-network-isolation-attaching.md`

Container networking unit

Attach services to user-defined bridge isolating workloads; practise detaching dependency causing outages.

Source: `09-container-networking-service-communication/02-internal-dns-hostname-resolution.md`

Container networking unit

Prefer hostname `postgres` / `redis` over brittle hard-coded bridge IPs diagnosing miswired env vars.

Source: `09-container-networking-service-communication/03-nat-and-port-publish-semantics.md`

Container networking unit

Map host `-p host:container` flows versus internal-only ClusterIP-esque thinking bridging future Kubernetes analogies.

Source: `09-container-networking-service-communication/04-localhost-inside-container-antipatterns.md`

Container networking unit

Demonstrate DATABASE_URL redis host pitfalls using `localhost` referencing self not sibling.

Source: `09-container-networking-service-communication/05-overlay-multi-host-concept.md`

Container networking unit

Read-only conceptual skim multi-node overlay prepping cluster networking without premature depth.

Source: `09-container-networking-service-communication/06-nginx-reverse-proxy-ingress-shape.md`

Container networking unit

External client→reverse proxy→app mental model practising broken upstream recovery.

Source: `09-container-networking-service-communication/07-tcpdump-ss-curl-troubleshooting.md`

Container networking unit

Packet/socket introspection ethically on lab traffic isolating DNS, firewall, stale connection symptoms.

Source: `09-container-networking-service-communication/08-stack-integration-dns-nat-proxy.md`

Container networking unit

Raise Symfony+Nginx+Redis+Postgres on custom bridge degrade DNS/NAT/hostnames/ports journaling recoveries.

Phase — 10-kubernetes-orchestration-cluster-lifecycle

Directory: `10-kubernetes-orchestration-cluster-lifecycle/`

Source: `10-kubernetes-orchestration-cluster-lifecycle/01-pod-and-kubectl-foundation.md`

Kubernetes orchestration unit

Pods vs VM confusion avoided; practise kubectl get/describe/logs/exec; corrupt image tug rehearse ethically.

Source: `10-kubernetes-orchestration-cluster-lifecycle/02-deployment-replica-rollout.md`

Kubernetes orchestration unit

Deployment→ReplicaSet→Pod chain; practise scale, rollout restart, bad image rollback narrative.

Source: `10-kubernetes-orchestration-cluster-lifecycle/03-service-discovery-selectors.md`

Kubernetes orchestration unit

ClusterIP/NodePort; break Service selector mismatches diagnosing DNS stale assumptions.

Source: `10-kubernetes-orchestration-cluster-lifecycle/04-namespace-configmap-secret.md`

Kubernetes orchestration unit

Split dev/stage/prod; mount ConfigMaps vs Secrets cautiously illustrating Symfony JWT secret hygiene redacted journaling.

Source: `10-kubernetes-orchestration-cluster-lifecycle/05-persistentvolume-storage-postgres.md`

Kubernetes orchestration unit

PVC + storage class rehearsal; validate data survives Pod deletion ethically with disposable volumes.

Source: `10-kubernetes-orchestration-cluster-lifecycle/06-ingress-routing-flow.md`

Kubernetes orchestration unit

Ingress bridging edge to Services; degrade rules observe 404/backends mismatch recovery.

Source: `10-kubernetes-orchestration-cluster-lifecycle/07-probes-and-resource-limits.md`

Kubernetes orchestration unit

Liveness/readiness + CPU/mem limits practising overload behavioural differences observably ethically.

Source: `10-kubernetes-orchestration-cluster-lifecycle/08-horizontal-pod-autoscaling-intro.md`

Kubernetes orchestration unit

HPA synthetic load etiquette only on authorised tiny clusters disclaim resource honesty.

Source: `10-kubernetes-orchestration-cluster-lifecycle/09-rbac-serviceaccount-least-privilege.md`

Kubernetes orchestration unit

Role/RoleBinding exercises denial then calibrated allow reflecting least privilege narratives.

Source: `10-kubernetes-orchestration-cluster-lifecycle/10-affinity-taints-tolerations.md`

Kubernetes orchestration unit

Scheduler steering stories—affinities vs taints/tolerations illustrative within lab constraints acknowledging k3d simplifications.

Source: `10-kubernetes-orchestration-cluster-lifecycle/11-cluster-troubleshooting-correlation.md`

Kubernetes orchestration unit

CrashLoop DNS mounts ingress permission systematic triage scripted fault injections.

Source: `10-kubernetes-orchestration-cluster-lifecycle/12-full-stack-integration-symfony-data-plane.md`

Kubernetes orchestration unit

Symfony+Redis+Postgres+Go+Ingress stacked manifest rehearsal degrade documentation marathon.

Source: `10-kubernetes-orchestration-cluster-lifecycle/13-pod-disruption-budgets-and-controlled-drain.md`

Unit 13 — PodDisruptionBudgets and considerate drains

PDBs translate human availability commitments into scheduler-aware constraints guarding voluntary disruptions (node drains, rollout surges). They do not exempt you from buggy applications—yet they curb thundering herd surprises during upgrades responsibly.

Laboratory story: declare a conservative `maxUnavailable` for a StatefulSet or Deployment handling sessions; combine with `kubectl drain` respecting PDB denials ethically on disposable clusters only conscientiously ethically.

When PDB blocks maintenance, practise communication pattern: quantify risk trade-off, temporary relaxations with peer review—not silent deletion of guarding objects irresponsibly thoughtlessly ethically.

Source: `10-kubernetes-orchestration-cluster-lifecycle/14-resource-quotas-and-limit-ranges.md`

Unit 14 — Namespace quotas & LimitRanges cooperative fairness

Multi-tenant clusters lean on ResourceQuota enforcing aggregate CPU/mem/object counts plus LimitRanges nudging sane defaults preventing single namespace starving neighbours unknowingly ethically.

Create dev/stage namespaces in k3d: apply quotas, exceed them deliberately reading `kubectl describe quota` frustrations mapping product engineer complaints empathetically for future platform interactions responsibly.

Tie organisational theme: PDB + quota interplay—overly tiny quotas may falsely trigger autoscaler starvation thrash ethically observed micro-lab scale only ethically.

Source: `10-kubernetes-orchestration-cluster-lifecycle/15-network-policies-layered-micro-segmentation.md`

Unit 15 — NetworkPolicy introductions (Calico/Cilium-class mental model)

NetworkPolicies express allow/deny traffic between Pods and namespaces—implementing coarse micro-segmentation beyond implicit open east-west chatter defaults Kubernetes historically tolerated naively ethically.

When CNI forbids Policies (some minimal labs), simulate mental exercise reading YAML guarding Symfony→Postgres lanes only—not blanket allow-all ethically documented limitations honestly ethically.

Troubleshooting matrix: symmetric label selectors, egress DNS to kube-dns, unintended headless vs cluster IP confusion referencing prior `07-*` foundations bridging consciously ethically.

Source: `10-kubernetes-orchestration-cluster-lifecycle/16-statefulset-and-volume-patterns-deepening.md`

Unit 16 — StatefulSet operations & resilient storage idioms

Deployments suit ephemeral web tiers; StatefulSets orchestrate replicas with predictable network identities & sticky volumes—for databases or queues you intentionally self-manage ethically documenting risks versus RDS/ElastiCache managed counterparts consciously thoughtfully.

Laboratory rehearsal: Postgres via StatefulSet aligning PVC templates; practise ordered rollout nuances & headless Services DNS entries correlating StatefulSet identity responsibly disposable only ethically thoughtfully.

Surgical upgrade narrative: correlate backup (`20-*`), readiness probes tuning, PDB interplay—simulate controlled failure journaling responsibly ethically.

Source: `10-kubernetes-orchestration-cluster-lifecycle/17-cluster-lifecycle-upgrade-and-deprecation-awareness.md`

Unit 17 — Cluster upgrades & API deprecation vigilance

Professional Kubernetes operators reconcile distributor release notes: removed beta APIs, container runtime transitions, etcd backup cadence—even if partially abstracted managed control planes shift responsibilities sideways not away intentionally consciously.

Maintain personal checklist: staging upgrade rehearsal, conformance smoke tests (`kubectl`-level), Ingress controller compatibility pinning, CSI driver upgrade coupling responsibly ethically conscientiously ethically.

Tie Helm (`09-*`) & Terraform/EKS narratives (`19-*`) when cloud control plane rotates beneath you unexpectedly—coordinate version skew windows consciously consciously ethically conscientiously ethically.

Phase — 11-helm-kubernetes-packaging-deployment

Directory: `11-helm-kubernetes-packaging-deployment/`

Source: `11-helm-kubernetes-packaging-deployment/01-chart-structure-helm-create.md`

Helm packaging unit

Chart.yaml templates/values separation mental model scaffolding first chart ethically.

Source: `11-helm-kubernetes-packaging-deployment/02-values-driven-configuration.md`

Helm packaging unit

Drive replicas image tag ports exclusively via values practising anti-hardcode discipline.

Source: `11-helm-kubernetes-packaging-deployment/03-templates-and-helm-template-render.md`

Helm packaging unit

Templating hygiene using helm template inspecting rendered manifests consciously before apply.

Source: `11-helm-kubernetes-packaging-deployment/04-realistic-symfony-packaging.md`

Helm packaging unit

Bundle Deployment Service ConfigMap Secret illustrating separation of configurable vs confidential.

Source: `11-helm-kubernetes-packaging-deployment/05-chart-dependencies.md`

Helm packaging unit

Upstream dependency charts chaining Redis Postgres responsibly version pinning awareness.

Source: `11-helm-kubernetes-packaging-deployment/06-upgrade-release-lifecycle.md`

Helm packaging unit

Iterative helm upgrade rehearsals narrating immutable release lineage observability minimally.

Source: `11-helm-kubernetes-packaging-deployment/07-rollback-history.md`

Helm packaging unit

Leverage helm history rollback after deliberate misconfiguration appreciating safe regression velocity.

Source: `11-helm-kubernetes-packaging-deployment/08-helm-troubleshooting.md`

Helm packaging unit

Mis-set values/secrets/port skew debugging via helm status get values dry-run mentalities responsibly.

Source: `11-helm-kubernetes-packaging-deployment/09-helm-integration-lab-full-stack.md`

Helm packaging unit

Compose prior lessons across Symfony Go data stores ingress degrade marathon documentation.

Source: `11-helm-kubernetes-packaging-deployment/10-values-schema-validation-and-contracts.md`

Unit 10 — `values.schema.json` and consumer contracts

Charts exploding optional booleans silently produce invalid manifests discovered only runtime—professional charts publish JSON schemas documenting accepted fields and defaults preventing malformed releases early responsibly consciously.

Laboratory adaptation: scaffold minimal schema guarding image repository/tag/replica combos; practise `helm template` failing loudly on purposely invalid CI inputs responsibly ethically conscientiously ethically.

Tie organisational CI: lint chart schema analogous to PHPUnit—fail merges before destructive apply attempts ethically responsibly consciously conscientiously ethically.

Source: `11-helm-kubernetes-packaging-deployment/11-hooks-and-tests-conscious-automation.md`

Unit 11 — Helm hooks & tests harness without surprise outages

Hooks automate pre/post-upgrade jobs (DB migrations)—yet naive hooks extend outage windows unpredictable if jobs wedge mysteriously ethically rehearse TTL & backoff discipline consciously ethically.

Charts may ship tests (`helm test`), mirroring synthetic smoke validations post-install—articulate coupling with Prometheus blackbox exporters vs chart-local job patterns consciously ethically.

Document rollback interplay: aborted hook leaving partial schema mutation demands forward-fix playbook narrative rehearsal responsibly ethically conscientiously ethically.

Source: `11-helm-kubernetes-packaging-deployment/12-helm-oci-registry-and-chart-publishing-flow.md`

Unit 12 — Charts as OCI artefacts

Publishing charts beside images into OCI registries streamlines versioning & RBAC aligning container supply chains—particularly when GitOps controllers fetch charts identically artefacts fetch images ethically documented vendor specifics consciously consciously ethically.

Laboratory analogue: replicate push/pull using authenticated registry sandbox you own ethically; annotate differences versus classic `helm repo index` tarball hosting consciously ethically.

Security interplay: impersonation tokens, robotic CI promotion lanes only after vulnerability scans align `18-*` expectations consciously ethically responsibly consciously ethically.

Source: `11-helm-kubernetes-packaging-deployment/13-gitops-and-release-controller-framing.md`

Unit 13 — GitOps & release-controller mental alignment

Helm often pairs with Flux/Argo CD-class controllers syncing Git-declared releases—professional operators reason about reconcile loops, drift detection, rollback semantics harmonising chart versions with cluster truth consciously ethically conscientiously ethically.

Laboratory light-weight: practise manual Git→ `helm upgrade` choreography echoing reconcile cadence—even before adopting full GitOps ethically documenting conceptual mapping consciously ethically.

Pitfalls: conflicting imperative `kubectl` edits battling declarative reconcile loops—establish team rule-of-law consciously consciously ethically ethically.

Phase — 12-terraform-infrastructure-lifecycle

Directory: `12-terraform-infrastructure-lifecycle/`

Source: `12-terraform-infrastructure-lifecycle/01-provider-init-plan-apply.md`

Terraform IaC unit

Declarative model with docker or lightweight provider lab illustrating init/plan/apply destroy discipline ethically disposable resources.

Source: `12-terraform-infrastructure-lifecycle/02-variables-and-outputs.md`

Terraform IaC unit

Parameterising configuration surfacing IPs endpoints via outputs for downstream choreography consciously.

Source: `12-terraform-infrastructure-lifecycle/03-state-and-drift.md`

Terraform IaC unit

Observe terraform state divergence after manual deletes practising refresh reconciliation honesty.

Source: `12-terraform-infrastructure-lifecycle/04-dependency-graph.md`

Terraform IaC unit

Visualise implicit ordering constraints network preceding compute preceding app attachment storytelling.

Source: `12-terraform-infrastructure-lifecycle/05-modules-refactor-duplication-out.md`

Terraform IaC unit

Factor repeated network/compute motifs into reusable module boundaries consciously scoped.

Source: `12-terraform-infrastructure-lifecycle/06-remote-state-and-locking.md`

Terraform IaC unit

Articulate communal state locking preventing concurrent apply clashes referencing S3+Dynamo illustrative reading later AWS phase.

Source: `12-terraform-infrastructure-lifecycle/07-workspaces-or-equivalent-environment-split.md`

Terraform IaC unit

Dev/stage/prod separation strategies noting workspace ergonomics caveats aligning organisational standards.

Source: `12-terraform-infrastructure-lifecycle/08-kubernetes-provider-crossover.md`

Terraform IaC unit

Terraform applying namespace/deployment scaffolding bridging infra vs workload ownership cautiously ethically disposable cluster.

Source: `12-terraform-infrastructure-lifecycle/09-stack-network-vm-k8-integration.md`

Terraform IaC unit

Layered provisioning narrative stitching components respecting dependency graph journaling.

Source: `12-terraform-infrastructure-lifecycle/10-troubleshooting-refresh-and-recovery.md`

Terraform IaC unit

Simulate variable corruption dependency skew recovery via plan discipline documentation.

Source: `12-terraform-infrastructure-lifecycle/11-integration-provisioning-rollout-drill.md`

Terraform IaC unit

Long-form solo rehearsal combining modules outputs remote backends hypothetically ethically.

Source: `12-terraform-infrastructure-lifecycle/12-aws-vpc-ec2-rds-outline.md`

Terraform IaC unit

High-level VPC subnet SG EC2 RDS pattern sketch deferring deep IAM cost optimisation until dedicated cloud track—articulate sequencing honestly reading official guides while practising safely in scratch accounts only.

Source: `12-terraform-infrastructure-lifecycle/13-remote-backend-hardening-and-locking-realities.md`

Unit 13 — Remote backends & locking realism

Teams graduate from local `.tfstate` to remote stores (S3+locking, Terraform Cloud workspaces) guaranteeing collaborative apply coherence—understand SSE encryption, versioning, IAM scoping thoughtfully consciously ethically conscientiously ethically.

Laboratory: configure toy remote backend (ethically segregated sandbox) observing lock contention behaviours when simultaneous applies collide consciously documenting compassionate peer communications consciously ethically ethically.

Pitfalls: orphaned locks after CI abort—articulate unlocking ceremony responsibilities guard-railed with audit consciously ethically ethically.

Source: `12-terraform-infrastructure-lifecycle/14-ci-pipelines-for-plan-and-apply-segregation.md`

Unit 14 — CI segregation: plan vs guarded apply

Professional infra applies split speculative `terraform plan` reviews from narrowed apply roles enforcing branch protections & manual approvals ethically mirroring seriousness production demands consciously ethically.

Compose GitLab/GitHub Actions sketch illustrating OIDC ephemeral credentials rather long-lived IAM user keys ethically emphasising posture improvement narratives consciously ethically.

Discuss blast-radius bounding: targeted `-target` uses sparing emergencies only—not daily habit ethically documenting operational debt consciously ethically.

Source: `12-terraform-infrastructure-lifecycle/15-policy-as-code-basics-and-boundaries.md`

Unit 15 — Policy-as-code (Sentinel / OPA / native guardrails sketch)

Governance attaches automated policy checks rejecting plans violating tagging, encryption toggles, or network openness—articulate interplay with organisational risk appetite without implying one vendor universal ethically consciously ethically.

Laboratory may stub `terraform validate` custom checks simpler first before enterprise policy stacks arrive consciously ethically.

When policy denies legitimate emergency change, escalate human exception workflow consciously consciously ethically ethically.

Source: `12-terraform-infrastructure-lifecycle/16-drift-detection-and-environment-reconciliation.md`

Unit 16 — Drift detection & reconciliation choreography

Declarative infra diverges reality after manual console hotfixes drift detection pipelines surface—professional teams schedule systematic reconciliations or import discipline consciously ethically.

Recreate purposeful drift ethically disposable stack: reconcile via `refresh` narratives & import remediation choices consciously ethically.

Bridging mindset: analogous GitOps reconcile loops share philosophical lineage consciously consciously ethically ethically.

Source: `12-terraform-infrastructure-lifecycle/17-multi-environment-and-module-versioning-strategy.md`

Unit 17 — Multi-environment rollout & semver module pinning

Modules versioned via Git tags/source pins prevent silent surprise upgrades cascading environments—establish promotion pipeline dev→stage→prod with semantic module constraints consciously ethically.

Tie workspaces (`07-*`) or directory-per-env strategies consciously selecting organisational consistent story—not hybrid chaos unconsciously ethically.

Operational note: coupling application Helm releases (`09-*`) with infra module bumps demands coordinated blackout windows conscientiously ethically.

Source: `12-terraform-infrastructure-lifecycle/18-localstack-aws-local-testing.md`

Terraform — `18-localstack-aws-local-testing`

Focus: Run AWS services locally with LocalStack so you can test Terraform plans and apply cycles without touching a real AWS account or incurring cost.

Practise focus

- Install LocalStack via Docker Compose and confirm `awslocal` CLI resolves to `localhost:4566`
- Configure Terraform AWS provider to point at LocalStack endpoint (`endpoint` overrides per service)
- Create S3 bucket, SQS queue, and IAM policy against LocalStack; run `plan → apply → destroy` cycle
- Validate state file reflects local resources identically to real AWS flow
- Understand LocalStack free tier limits vs Pro (which services need Pro licence)
- Use LocalStack for CI pipeline dry-runs — catch resource misconfigurations before they hit staging
- Teardown and reset LocalStack state between test runs

Source: `12-terraform-infrastructure-lifecycle/19-terraform-localstack-integration.md`

Terraform — `19-terraform-localstack-integration`

Focus: Wire a real Terraform module (VPC + EC2 + RDS shape) to LocalStack and validate the full apply/destroy lifecycle locally before promoting to real AWS.

Practise focus

- Parameterise AWS provider with a variable flag (`local_mode = true`) that switches endpoints to LocalStack vs real AWS
- Apply the VPC + subnets + security group module against LocalStack; inspect created resources via `awslocal ec2 describe-vpcs`
- Simulate drift: manually delete a resource in LocalStack, run `terraform refresh`, observe plan diff
- Run `terraform plan` in CI against LocalStack as a mandatory gate before any `apply` reaches staging
- Debug provider errors specific to LocalStack (partial service support, eventual consistency quirks)
- Document the handoff boundary: what LocalStack validates vs what only real AWS can catch (IAM propagation delays, quota limits)

Source: `12-terraform-infrastructure-lifecycle/20-cdk-typescript-aws-native-iac.md`

Terraform — `20-cdk-typescript-aws-native-iac`

Focus: Understand AWS CDK as an alternative IaC surface — write infrastructure in TypeScript, synthesise to CloudFormation, and know when CDK fits over Terraform.

Practise focus

- Bootstrap a CDK app: `cdk init app --language typescript`, understand `App → Stack → Construct` hierarchy
- Define an S3 bucket and Lambda function in CDK; run `cdk synth` and read the generated CloudFormation template
- `cdk deploy` to a real AWS account; observe CloudFormation stack creation in the console
- Understand L1 (raw CFN), L2 (opinionated constructs), L3 (patterns) abstraction levels
- Compare CDK vs Terraform: CDK wins on AWS-native constructs and type safety; Terraform wins on multi-cloud and state portability
- When to use both together: Terraform for foundational infra (VPC, IAM), CDK for application-level AWS resources
- `cdk diff` as the CDK equivalent of `terraform plan`
- Destroy stack cleanly: `cdk destroy`

Phase — 13-ansible-configuration-management

Directory: `13-ansible-configuration-management/`

Source: `13-ansible-configuration-management/01-ansible-mental-model-and-why-it-exists.md`

Ansible — `01-ansible-mental-model-and-why-it-exists`

Focus: Understand where Ansible sits in the toolchain — Terraform creates infrastructure, Ansible configures what runs on it. Agentless push model via SSH.

Practise focus

- Distinguish configuration management (Ansible) from infrastructure provisioning (Terraform) — the handoff boundary
- Understand the push model: control node SSHes into managed nodes, no agent required
- Install Ansible on a control node; confirm `ansible --version` and Python dependency chain
- Run first ad-hoc command: `ansible all -i hosts -m ping`
- Understand idempotency as the core contract — running a playbook twice must produce the same state
- Compare Ansible vs Chef/Puppet/Salt — why Ansible won on simplicity and agentless architecture
- Identify real-world use cases: EC2 post-provision setup, config drift remediation, patch management

---

Source: `13-ansible-configuration-management/02-inventory-static-and-dynamic.md`

Ansible — `02-inventory-static-and-dynamic`

Focus: Define and manage inventories — static INI/YAML files for fixed infrastructure, dynamic inventory plugins for AWS EC2 where IPs change on every deploy.

Practise focus

- Write a static `inventory.ini` with groups (`[webservers]` , `[databases]`) and host variables
- Convert to YAML inventory format; understand `host_vars/` and `group_vars/` directory conventions
- Install and configure the `amazon.aws.ec2` dynamic inventory plugin; authenticate via IAM role or credentials
- Run `ansible-inventory --list` to inspect resolved host list from AWS
- Filter dynamic inventory by EC2 tags (`Environment=staging` , `Role=api`) — target only relevant instances
- Understand `all` and `ungrouped` built-in groups; compose nested group hierarchies
- Test inventory with `ansible all -m ping` against a real or LocalStack-backed EC2 mock

---

Source: `13-ansible-configuration-management/03-playbooks-tasks-and-modules.md`

Ansible — `03-playbooks-tasks-and-modules`

Focus: Write playbooks that install packages, manage files, configure services — using the most common modules you will touch on every real job.

Practise focus

- Write a playbook that: installs nginx (`ansible.builtin.apt`), copies a config file (`ansible.builtin.copy` / `template`), and starts the service (`ansible.builtin.service`)
- Understand play structure: `hosts` , `become` , `vars` , `tasks` , `handlers`
- Use `handlers` for service restarts — trigger only on config change, not on every run
- Apply conditionals with `when` : skip a task if OS family is not Debian
- Loop over a list of packages with `loop` / `with_items`
- Register task output with `register` and use it in subsequent tasks
- Run playbook with `--check` (dry-run) and `--diff` to preview changes before applying
- Common modules to master: `apt` , `yum` , `copy` , `template` , `file` , `lineinfile` , `service` , `user` , `command` , `shell`

Source: `13-ansible-configuration-management/04-roles-structure-and-reuse.md`

Ansible — `04-roles-structure-and-reuse`

Focus: Organise playbook logic into roles — reusable, testable units with a conventional directory structure. Roles are how real teams share and version Ansible code.

Practise focus

- Generate a role scaffold with `ansible-galaxy init myrole`; understand `tasks/`, `handlers/`, `templates/`, `files/`, `vars/`, `defaults/`, `meta/`
- Difference between `vars/` (high priority, not easily overridden) and `defaults/` (low priority, meant to be overridden)
- Write a `nginx` role: install, configure with a Jinja2 template, enable and start
- Apply the role in a playbook via `roles:` key; override defaults with `role_params`
- Install a community role from Ansible Galaxy: `ansible-galaxy install geerlingguy.docker`
- Pin role versions in `requirements.yml`; install with `ansible-galaxy install -r requirements.yml`
- Understand role dependencies via `meta/main.yml`
- Structure a real project: `site.yml` → `roles/` → `group_vars/` → `inventory/`

Source: `13-ansible-configuration-management/05-variables-templates-and-vault.md`

Ansible — `05-variables-templates-and-vault`

Focus: Manage configuration data cleanly — Jinja2 templates for dynamic files, variable precedence hierarchy, and Ansible Vault for secrets.

Practise focus

- Write a Jinja2 template (`nginx.conf.j2`) with variable interpolation and conditional blocks; deploy with `template` module
- Understand variable precedence order (22 levels) — know that `extra_vars` (`-e`) always wins, `defaults/` always loses
- Define environment-specific vars in `group_vars/staging/` vs `group_vars/production/`
- Encrypt a secrets file with `ansible-vault encrypt secrets.yml`; decrypt inline during playbook run with `--ask-vault-pass` or `--vault-password-file`
- Use `ansible-vault encrypt_string` to inline-encrypt a single variable value in a vars file
- Rotate vault password: decrypt all, re-encrypt with new password
- Never commit plaintext secrets — enforce vault usage in CI with a pre-commit hook

---

Source: `13-ansible-configuration-management/06-aws-ec2-provisioning-and-dynamic-fleet.md`

Ansible — `06-aws-ec2-provisioning-and-dynamic-fleet`

Focus: Use Ansible to configure EC2 instances after Terraform creates them — the standard post-provisioning handoff pattern for AWS fleets.

Practise focus

- Use `amazon.aws.ec2_instance` module to launch an EC2 instance from Ansible (understand when this vs Terraform is appropriate)
- More common pattern: Terraform outputs instance IPs → write to dynamic inventory → Ansible picks up and configures
- Implement the handoff: Terraform `output "instance_ips"` → local-exec writes `inventory/hosts` → `ansible-playbook site.yml -i inventory/`
- Configure a freshly launched EC2: install Docker, configure logging, set up a systemd service
- Use EC2 tags as inventory groups via dynamic inventory plugin — no static IP management
- Handle SSH key injection: Terraform places keypair, Ansible uses `ansible_ssh_private_key_file`
- Run Ansible from GitLab CI after `terraform apply` succeeds — full automated provision + configure pipeline

---

Source: `13-ansible-configuration-management/07-ansible-in-cicd-and-gitlab-integration.md`

Ansible — `07-ansible-in-cicd-and-gitlab-integration`

Focus: Run Ansible playbooks from GitLab CI — authenticated, vault-aware, and scoped to the right environment via protected variables.

Practise focus

- Add an `ansible` stage to `.gitlab-ci.yml` after the `deploy` (Terraform) stage
- Install Ansible in a CI job using a Docker image (`cytopia/ansible` or build your own)
- Inject vault password via GitLab CI protected variable (`ANSIBLE_VAULT_PASSWORD`); write to temp file, clean up after run
- Use `ansible-lint` as a quality gate job — fail pipeline on playbook violations
- Scope playbook execution to environment: `staging` branch runs against staging inventory, `main` against production
- Use GitLab environments to track which Ansible version last ran against each host group
- Cache `ansible-galaxy install` output as a CI artifact to avoid re-downloading roles on every run
- Debug failed CI runs: `ansible-playbook -vvv` output in job logs, interpreting SSH errors vs task failures

---

Source: `13-ansible-configuration-management/08-drift-detection-and-config-remediation.md`

Ansible — `08-drift-detection-and-config-remediation`

Focus: Detect and correct configuration drift on live servers — the operational superpower that justifies Ansible alongside Terraform and Kubernetes.

Practise focus

- Run a playbook in `--check` mode on a production host to surface drift without making changes
- Schedule nightly drift detection in CI: playbook runs in check mode, fails pipeline if drift detected, pages on-call
- Simulate drift: manually change nginx config on a server; re-run playbook and confirm it corrects back to desired state
- Use `assert` module to validate invariants (file exists, service is running, port is open) as a health-check playbook
- Understand the limits of idempotency: `command` / `shell` modules are not idempotent by default — use `creates` / `removes` guards or prefer purpose-built modules
- Distinguish Ansible remediation (immediate SSH-push fix) vs Terraform remediation (state reconciliation) — choose based on what drifted
- Build a simple audit playbook: check OS patch level, verify no unauthorized users, confirm firewall rules — output a report

---

Source: `13-ansible-configuration-management/09-terraform-ansible-full-stack-integration.md`

Ansible — `09-terraform-ansible-full-stack-integration`

Focus: End-to-end lab — Terraform provisions AWS infrastructure, Ansible configures application layer, GitLab CI orchestrates the full pipeline.

Practise focus

- Terraform creates: VPC, public subnet, security group, EC2 instance (with keypair), outputs instance public IP
- Terraform `local-exec` provisioner writes IP to `ansible/inventory/hosts` after apply (or use Terraform output + CI step)
- Ansible playbook runs post-Terraform: installs Docker, copies application config via template (with vault-managed DB password), starts application container
- GitLab CI pipeline: `terraform plan` → approval gate → `terraform apply` → `ansible-playbook site.yml` → smoke test (curl health endpoint)
- Handle failures: Terraform apply fails → Ansible never runs; Ansible fails → instance exists but is unconfigured → fix and re-run Ansible only
- Teardown: `terraform destroy` cleans infrastructure; no Ansible cleanup needed (infra gone = config gone)
- Reflect on the boundary: Terraform owns infrastructure lifecycle, Ansible owns software configuration lifecycle — neither crosses into the other's domain

---

Phase — 14-gitlab-cicd-delivery-deployment

Directory: `14-gitlab-cicd-delivery-deployment/`

Source: `14-gitlab-cicd-delivery-deployment/01-first-pipeline-stages-commit-to-result.md`

01 First Pipeline Stages Commit To Result — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Introduce minimalist `.gitlab-ci.yml` illustrating commit→pipeline→job→result narration; align PHPUnit invocation with your Symfony paths.

Source: `14-gitlab-cicd-delivery-deployment/02-gitlab-runner-execution-surfaces.md`

02 Gitlab Runner Execution Surfaces — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Install GitLab Runner; compare docker executor vs shell; rehearse degraded runner/offline/perms/Docker socket failures and recovery.

Source: `14-gitlab-cicd-delivery-deployment/03-cache-and-artifacts-optimisation.md`

03 Cache And Artifacts Optimisation — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Use `cache` (e.g. vendor) and `artifacts` between jobs; practise missing artefact breakage and remediation.

Source: `14-gitlab-cicd-delivery-deployment/04-quality-gates-tests-lint-phpstan.md`

04 Quality Gates Tests Lint Phpstan — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Stages: tests then quality with PHPUnit + lint + PHPStan; inject deliberate PHP and static-analysis faults.

Source: `14-gitlab-cicd-delivery-deployment/05-docker-build-in-pipeline.md`

05 Docker Build In Pipeline — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Pipeline builds image after gates; break Dockerfile expecting failed promotion.

Source: `14-gitlab-cicd-delivery-deployment/06-security-scan-trivy-image.md`

06 Security Scan Trivy Image — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Add image scan step (e.g. Trivy); observe vulnerable-layer reporting and fix path.

Source: `14-gitlab-cicd-delivery-deployment/07-environments-protected-branches.md`

07 Environments Protected Branches — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Define environment naming; block deploy from feature branches; allow controlled promote from protected main.

Source: `14-gitlab-cicd-delivery-deployment/08-deployment-job-automation.md`

08 Deployment Job Automation — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Chain to deploy surrogate (local/stack you own) after scan; narrate lifecycle vs manual babysitting.

Source: `14-gitlab-cicd-delivery-deployment/09-rollback-and-manual-recovery-jobs.md`

09 Rollback And Manual Recovery Jobs — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Deploy bad artefact rehearsal; practise manual rollback / redeploy stable tag discipline.

Source: `14-gitlab-cicd-delivery-deployment/10-full-pipeline-integration-fault-marathon.md`

10 Full Pipeline Integration Fault Marathon — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Full stack rehearsal: push→tests→lint→phpstan→docker build→scan→deploy—rotate faults (runner, lint, Docker, deploy) and document.

Phase — 15-devsecops-supply-chain-secure-ci

Directory: `15-devsecops-supply-chain-secure-ci/`

Source: `15-devsecops-supply-chain-secure-ci/01-shift-left-security-culture.md`

Security continuous across lifecycle

Outcome mindset: Articulate integrating checks before deploy—not mythical final gate auditor heroics lone Friday evening.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/02-composer-module-and-go-mod-scans.md`

Dependency scanning ergonomics

Outcome mindset: Run dependency CVE scans on Composer/Go modules; remediate illustrative vulnerable package responsibly using disposable forks.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/03-container-image-scanning-trivy.md`

Registry-bound image scrutiny

Outcome mindset: Pipe built images through Trivy illustrating failing builds on catastrophic base layer CVE severity thresholds consciously humane.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/04-repository-secret-scan-gitleaks-pattern.md`

Secret scanning pipelines

Outcome mindset: Seed synthetic secret purposely removed afterwards verifying scanner prevents promotion—ethical destruction immediately post validation.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/05-sast-baseline-for-php-go.md`

Static analysis augmentation

Outcome mindset: Augment PHPUnit gates with deliberate code smells static analyzers catching—excluding perfectionist noise drowning signal.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/06-sbom-generation-and-review.md`

SBOM transparency

Outcome mindset: Produce bill-of-materials snapshot emphasising organisational traceability confronting supply-chain compromise headlines responsibly.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/07-gitlab-unified-security-pipeline-blocks.md`

Single pipeline quality+security choreography

Outcome mindset: Compose stage ordering: tests · lint/static · dependency/image/secret scanners · build · deploy ethically pausing merges on regressions thoughtfully.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/08-supply-chain-threat-model-talkthrough.md`

High-level attacker narrative arcs

Outcome mindset: Enumerate dependency poisoning, compromised base images, exposed pipeline credentials—without fearmongering absolving engineering laziness casually.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/09-incident-remediation-supply-chain-mindset.md`

Security incident rehearsal

Outcome mindset: Conduct tabletop on scanner bypass or leaked pipeline token—articulate revocation + rotation discipline succinctly ethically.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `15-devsecops-supply-chain-secure-ci/10-integration-devsecops-capstone-gitlab-docker.md`

Integrated security gate capstone

Outcome mindset: Unify Symfony+Go Dockerfile pipeline with scanners toggled intentionally failing remediation loops documenting deltas responsibly.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Phase — 16-production-reliability-and-release-safety

Directory: `16-production-reliability-and-release-safety/`

Source: `16-production-reliability-and-release-safety/01-rollbacks-across-deploy-surfaces.md`

Rollback choreography across Helm, GitLab, and runtime toggles

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Contrast redeploy previous artefact vs forward-fix strategies responsibly articulating downtime trade-offs ethically.

Source: `16-production-reliability-and-release-safety/02-database-backups-and-restore-validation.md`

Backup scepticism pipeline

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Operate Postgres backup rehearsal emphasising restores-not-backups existential lesson responsibly destroying disposable dataset only ethically.

Source: `16-production-reliability-and-release-safety/03-disaster-recovery-tabletop-shape.md`

Disaster recovery narrative arcs

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Author concise recovery objectives RTO/RPO vocabulary qualitatively without enterprise consulting cosplay overshadowing practicality ethically.

Source: `16-production-reliability-and-release-safety/04-blue-green-rollout-shape.md`

Blue/green parallelism mental model

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Sketch parallel environment promotion toggling verifying health gates before draining old colour ethically on lab mocks only.

Source: `16-production-reliability-and-release-safety/05-canary-and-progressive-traffic-shift.md`

Canary / progressive exposure

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Discuss partial traffic bifurcation monitoring error budgets qualitatively referencing service mesh proxies or Ingress weighting ethically cautiously nuanced.

Source: `16-production-reliability-and-release-safety/06-postmortem-format-and-blameless-accountability.md`

Postmortem structure

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Produce incident retrospective template outlining impact timelines root causal factors preventative backlog accountability responsibly.

Source: `16-production-reliability-and-release-safety/07-slo-sla-awareness-for-operators.md`

SLO versus marketing SLA distinctions

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Converse meaning of availability targets thoughtfully acknowledging customer promises vs engineer error budget nuance ethically without numeric absolutism absent data.

Phase — 17-aws-cloud-infrastructure-and-operations

Directory: `17-aws-cloud-infrastructure-and-operations/`

Source: `17-aws-cloud-infrastructure-and-operations/01-iam-identities-policies-least-privilege.md`

IAM fundamentals + least-privilege rehearsals

Outcome mindset: IAM fundamentals + least-privilege rehearsals

Practise focus

- Enumerate identity vs permission vs resource mapping; revoke-over-grant tabletop ethically on scratch IAM users disposable.

Source: `17-aws-cloud-infrastructure-and-operations/02-ec2-security-groups-instance-access.md`

EC2 compute introduction

Outcome mindset: EC2 compute introduction

Practise focus

- Bootstrap Ubuntu instance practising restricted Security Group deltas; articulate key hygiene vs ephemeral session managers high-level ethically.

Source: `17-aws-cloud-infrastructure-and-operations/03-vpc-subnets-and-routing.md`

VPC topology literacy

Outcome mindset: VPC topology literacy

Practise focus

- Contrast public/private subnet roles, IGW versus NAT gateways, route table debugging narratives responsibly cost-aware ethically.

Source: `17-aws-cloud-infrastructure-and-operations/04-rds-managed-postgresql-access-shape.md`

Managed relational posture

Outcome mindset: Managed relational posture

Practise focus

- Exercise SG-restricted DB connectivity from app tier; forbid wide-open Postgres anti-pattern horror stories ethically.

Source: `17-aws-cloud-infrastructure-and-operations/05-s3-object-storage-and-policies.md`

S3 ergonomics distinguishing object semantics

Outcome mindset: S3 ergonomics distinguishing object semantics

Practise focus

- Demonstrate versioning/lifecycle interplay cautiously aligning compliance unknowns disclaimers ethically.

Source: `17-aws-cloud-infrastructure-and-operations/06-route53-alb-and-target-health.md`

DNS pointing at load-balanced targets

Outcome mindset: DNS pointing at load-balanced targets

Practise focus

- Articulate layered health probing responsibilities across ALB target groups ethically rehearsing outage simulations disposable environments.

Source: `17-aws-cloud-infrastructure-and-operations/07-cloudwatch-metrics-and-alarms.md`

CloudWatch grounding

Outcome mindset: CloudWatch grounding

Practise focus

- Hook CPU-oriented synthetic alarm illustrative only—avoid blasting production paging endpoints inadvertently ethically.

Source: `17-aws-cloud-infrastructure-and-operations/08-cloud-secret-providers-intro.md`

Cloud secret distribution sketch

Outcome mindset: Cloud secret distribution sketch

Practise focus

- Discuss AWS secret manager injecting runtime credentials aligning rotation responsibilities organisationally ethically.

Source: `17-aws-cloud-infrastructure-and-operations/09-ecs-services-and-task-networking.md`

ECS container orchestration primer

Outcome mindset: ECS container orchestration primer

Practise focus

- Contrast task definitions vs services; practise tiny deploy illustrative acknowledging Fargate/EC2 trade depth deferred consciously.

Source: `17-aws-cloud-infrastructure-and-operations/10-eks-managed-kubernetes-bridge.md`

Managed Kubernetes bridging prior k3d

Outcome mindset: Managed Kubernetes bridging prior k3d

Practise focus

- EKS mental overlay referencing earlier Helm/Terraform curricula cross-linking ethically cost cautiously.

Source: `17-aws-cloud-infrastructure-and-operations/11-iam-policy-debug-security-tabletop.md`

Policy debugging tabletop

Outcome mindset: Policy debugging tabletop

Practise focus

- Exercise overly tight IAM provoking pipeline breakage diagnosing CloudTrail-esque audit signals high-level ethically read-only rehearsals.

Source: `17-aws-cloud-infrastructure-and-operations/12-lambda-serverless-fit-sketch.md`

Serverless applicability nuance shallow

Outcome mindset: Serverless applicability nuance shallow

Practise focus

- Demonstrate miniature function handler emphasising operational limits—not universal hammer fantasy ethically.

Source: `17-aws-cloud-infrastructure-and-operations/13-terraform-aws-topology-capstone-recap.md`

Terraform choreography revisiting IaC

Outcome mindset: Terraform choreography revisiting IaC

Practise focus

- Praise reproducible infra vs console-only drift using Terraform snippets responsibly disposable accounts only ethically.

Source: `17-aws-cloud-infrastructure-and-operations/14-private-tier-connectivity-deepening.md`

Private tier connectivity deepening

Outcome mindset: Private tier connectivity deepening

Practise focus

- Discuss ALB bridging internet to private subnets housing apps + RDS ethically sketching segmentation responsibilities explicitly.

Source: `17-aws-cloud-infrastructure-and-operations/15-aws-organizations-scps-and-cross-account-access.md`

Unit 15 — AWS Organizations, SCPs, and cross-account access patterns

Large AWS estates centralise guardrails through AWS Organizations and **Service Control Policies** (SCPs) constraining what member accounts can ever enable—even if an IAM policy inside the account looks permissive on paper. This is not `19-*` day-one material, but you must recognise the existence of control planes above single-account IAM when designing landing zones.

Pair SCP thinking with **cross-account IAM roles** (typically `sts:AssumeRole` from a shared tooling account) so CI/CD or human break-glass never stores long-lived access keys on laptops. Sketch a three-account mental model: `security`, `workloads`, `network`—even if your lab only simulates one account.

Verification habit: when a pipeline suddenly cannot create an S3 bucket, consider **implicit deny via SCP** before spending hours debugging IAM inline policies in isolation.

Source: `17-aws-cloud-infrastructure-and-operations/16-transit-gateway-and-multi-vpc-network-topology.md`

Unit 16 — Transit Gateway style multi-VPC connectivity (conceptual lab)

When services outgrow a single VPC, teams connect spoke VPCs through **AWS Transit Gateway** (or peering patterns for smaller footprints) so shared services (egress inspection, centralised logging, private API endpoints) remain reachable without duplicating NAT infrastructure everywhere.

You may not build a full TGW lab on Free Tier; still draw address plans showing **non-overlapping RFC1918 CIDRs**, route table priorities, and where **inspection VPCs** live. Compare mental model to Kubernetes overlay networking (`07-*` , `08-*`).

Failure mode to rehearse narratively: asymmetric routing after adding a new spoke—document how you would prove with VPC Flow Logs (conceptual) rather than guessing security groups first.

Source: `17-aws-cloud-infrastructure-and-operations/17-ecs-fargate-vs-eks-operational-trade-space.md`

Unit 17 — ECS/Fargate vs EKS: choosing a container platform on AWS

Amazon ECS (often with Fargate) packages task definitions, service discovery, and scaling with less moving parts than **EKS**, which brings the full Kubernetes operational surface you already practised in `08-*` / `09-*` . Neither is universally superior: choose based on team skills, desired extension points, and integration with existing GitOps stacks.

Exercise: document a decision matrix for a Symfony + Go API stack: who owns cluster upgrades, how Helm charts port, what observability agents you must run, and how CI promotes images. Even a one-page matrix is enough to prove structured thinking.

Cost lens: Fargate removes node management but can surprise at steady high CPU—pair with cost dashboards later (`20-*` reliability themes) before production promises.

Source: `17-aws-cloud-infrastructure-and-operations/18-edge-security-cloudfront-waf-and-acm-practices.md`

Unit 18 — Edge delivery: CloudFront, WAF, and ACM in practice

Public HTTP surfaces often terminate TLS at **CloudFront** or **Application Load Balancers** with certificates from **ACM**. Professional operators understand cache key behaviours, origin shielding, and when to enable **AWS WAF** managed rule groups versus custom rules—without enabling every rule aggressively blocking legitimate API clients.

Lab substitute if budget-constrained: keep using `12-*` Traefik/Nginx locally but map each feature (TLS cert auto-renewal, geo headers, request logging) to its AWS analogue in writing.

Incident tie-in: when users see sporadic 403s, differentiate WAF vs security group vs origin misconfiguration using edge access logs (conceptual workflow).

Source: `17-aws-cloud-infrastructure-and-operations/19-resilience-backup-and-dr-patterns-on-aws.md`

Unit 19 — Resilience, backup, and DR patterns (RDS, S3, snapshots)

Translate `20-*` reliability ideas into AWS primitives: automated **RDS snapshots** with tested restores, **S3 versioning** + lifecycle rules, **cross-region replication** only when RPO demands justify cost. Always script a restore rehearsal summary—even if the rehearsal happens quarterly in a real job.

Document RPO/RTO targets qualitatively for a demo ecommerce database: how many minutes of data loss are tolerable, which snapshot window satisfies that, and who approves emergency PITR spends.

Pair with Terraform (`10-*`) cautiously: removing `DeletionProtection` via IaC is a production foot-gun—use policy checks introduced in `10-*` trailing units.

Source: `17-aws-cloud-infrastructure-and-operations/20-cost-governance-tagging-and-quotas.md`

Unit 20 — Cost awareness, tagging, and service quotas

Professional AWS work includes **cost allocation tags**, **budgets + anomaly detection**, and understanding **service quotas** before large-scale load tests take down an API Gateway unexpectedly. This is not FinOps certification depth—just enough to partner credibly with finance peers.

Exercise: define a mandatory tag schema (`Environment` , `Owner` , `CostCenter`) you would enforce via SCP or IaC validators. Describe how Grafana/Prometheus billing exporters differ from AWS Cost Explorer (both useful, different fidelity).

When scaling EKS (`10-eks*` narratives), remind yourself control plane hourly charges plus NAT Gateway data processing often dominate guesses—surfacing early in architectures avoids shock.

Source: `17-aws-cloud-infrastructure-and-operations/21-hybrid-connectivity-vpn-and-direct-connect-overview.md`

Unit 21 — Hybrid connectivity: Site-to-Site VPN & Direct Connect concepts

Many enterprises attach AWS to on-prem Active Directory realms or legacy mainframes via **Site-to-Site VPN** tunnels or leased-line style **Direct Connect**. You may never provision DX in a learner account, yet architecture reviews demand vocabulary: BGP prefixes, failover tunnels, resilience expectations.

Laboratory alternative: emulate VPN concepts with overlapping route scenarios in diagrams, then correlate to Kubernetes clusters needing outbound access to corp HTTP proxies responsibly.

Troubleshooting pattern: asymmetric routing after corporate firewall updates—coordinate with netops using traceroute artefacts from earlier Linux (`02-*`) and container networking drills (`07-*`).

Source: `17-aws-cloud-infrastructure-and-operations/22-ecs-fargate-end-to-end-deploy-lab.md`

AWS — `22-ecs-fargate-end-to-end-deploy-lab`

Focus: Full hands-on deployment of a containerised application to ECS Fargate — from ECR image push to live HTTPS traffic through ALB and Route53.

Practise focus

- Push a Docker image to ECR: `docker build → docker tag → aws ecr get-login-password | docker login → docker push`
- Write a Task Definition (JSON or Terraform): container image, CPU/memory, port mappings, environment variables from Secrets Manager
- Create an ECS Cluster and Service with Fargate launch type; set desired count to 2
- Attach an Application Load Balancer target group; confirm health checks pass before traffic shifts
- Wire Route53 A-record alias to ALB DNS name; verify HTTPS via ACM certificate on ALB listener
- Observe a rolling deployment: update image tag, trigger new service deployment, watch old tasks drain
- Debug a failed task: inspect stopped task reason, CloudWatch Logs (`awslogs` driver), security group mismatches
- Scale service manually then configure Application Auto Scaling on CPU utilisation target
- Destroy cleanly: deregister task definition, delete service, drain and delete cluster

---

Source: `17-aws-cloud-infrastructure-and-operations/23-codepipeline-codedeploy-native-ci.md`

AWS — `23-codepipeline-codedeploy-native-ci`

Focus: Build an AWS-native CI/CD pipeline using CodePipeline + CodeBuild + CodeDeploy — the deployment surface you encounter in AWS-first organisations.

Practise focus

- Understand the three stages: Source (CodeCommit/GitHub) → Build (CodeBuild) → Deploy (CodeDeploy/ECS)
- Write a `buildspec.yml` : install phase, build phase (docker build + push to ECR), post-build phase (update task definition)
- Configure CodeBuild project with an IAM role scoped to ECR push and S3 artifact write
- Create a CodePipeline with GitHub v2 source connection; trigger pipeline on push to `main`
- Deploy to ECS using CodeDeploy blue/green: configure `AppSpec` for ECS task definition swap
- Observe blue/green traffic shift in ALB listener rules; verify rollback on failed health check
- Compare: CodePipeline vs GitLab CI for ECS deploy — native IAM integration vs flexibility

• Add a manual approval action between staging and production stages

Phase — 18-edge-reverse-proxy-and-traffic-control

Directory: `18-edge-reverse-proxy-and-traffic-control/`

Source: `18-edge-reverse-proxy-and-traffic-control/01-nginx-forwarding-upstream-shape.md`

Nginx request forwarding sanity

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Topo: browser → nginx → Symfony/Go; probe wrong upstream/port reversibly.
- Iterate proxy_pass ergonomics aligning socket paths responsibly.

Source: `18-edge-reverse-proxy-and-traffic-control/02-forwarding-headers-origin-trust-boundaries.md`

Proxy forwarding headers discipline

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Emit/consume X-Forwarded-For · X-Forwarded-Proto · Host; surface spoofing caveat behind untrusted hops.
- Laravel/Symfony request IP trusting strategy awareness high-level ethically.

Source: `18-edge-reverse-proxy-and-traffic-control/03-tls-termination-edge-to-plain-backend.md`

TLS termination choreography

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Terminate HTTPS outward; sane internal plaintext only within controlled trust envelopes you document honestly.
- `openssl s_client` validate chains; rehearse corrupted cert rollback.

Source: `18-edge-reverse-proxy-and-traffic-control/04-response-compression-gzip-tradeoffs.md`

Compression at reverse proxy tier

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Enable gzip/deflate thoughtfully—measure payload deltas; avoid double-compression nightmares with app layering.

Source: `18-edge-reverse-proxy-and-traffic-control/05-rate-limiting-and-abuse-guardrails.md`

Rate limiting ethos

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Shape limits protecting upstream saturation vs punishing legitimate bursts—simulate flood ethically on lab workloads only.

Source: `18-edge-reverse-proxy-and-traffic-control/06-load-balancing-upstreams-and-health.md`

Upstream pools + availability signalling

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Round-robin illustrative multi Symfony replicas; degrade single member validating resilience narratives.

Source: `18-edge-reverse-proxy-and-traffic-control/07-traefik-dynamic-discovery-middleware.md`

Traefik as modern programmable edge

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Docker/socket providers; HTTPS + middleware chaining (auth trims, redirects) on disposable stacks.

Source: `18-edge-reverse-proxy-and-traffic-control/08-timeouts-buffering-cache-intro.md`

Edge protective timeouts & buffering

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Tune proxy ↔ upstream timeouts guarding slow backends; skim cache layers—never cache authenticated surprises naively defaulting.

Source: `18-edge-reverse-proxy-and-traffic-control/09-edge-troubleshooting-matrix.md`

Systematic proxy edge debugging

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Correlate curl, access/error logs, TLS traces, LB target health—fault classes: timeouts, certs, LB 502 ladders.

Source: `18-edge-reverse-proxy-and-traffic-control/10-integration-traefik-nginx-symfony-go-postgres.md`

Full-chain integration rehearsal

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Internet-facing Traefik+Nginx(or single edge choice)—Symfony+Go API+Postgres layering TLS+rates+timeouts; scripted degradation journaling.

Phase — 19-monitoring-metrics-alerting-and-promql

Directory: `19-monitoring-metrics-alerting-and-promql/`

Source: `19-monitoring-metrics-alerting-and-promql/01-golden-signals-and-metric-thinking.md`

What to measure consciously

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Latency · traffic · errors · saturation mental anchors supersetting toy dashboards obsessing meaningless counters.

Source: `19-monitoring-metrics-alerting-and-promql/02-prometheus-scrape-target-lifecycle.md`

Pull-based Prometheus mental model

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Service discovery stubs; scrape interval interplay; practise miswired metrics endpoint diagnosing empty targets ethically.

Source: `19-monitoring-metrics-alerting-and-promql/03-application-metrics-from-symfony.md`

Symfony instrumentation surfacing behaviours

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Expose exemplar histograms/counters practising RED-flavoured stories on synthetic sluggish routes.

Source: `19-monitoring-metrics-alerting-and-promql/04-application-metrics-from-go-runtime.md`

Go instrumentation & runtime vistas

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Process metrics goroutines/mem; saturate goroutine illustrative workload observing ethical ceilings only on lab VMs.

Source: `19-monitoring-metrics-alerting-and-promql/05-node-and-infra-exporters.md`

Bridging bare-metal/container metrics exporters

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Correlate host exporter series with noisy neighbour containers—articulate cardinality caution.

Source: `19-monitoring-metrics-alerting-and-promql/06-label-taxonomy-and-cardinality-awareness.md`

Label discipline versus metric explosions

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Design selective labels sparing high-card combos; practise PromQL slicing safely.

Source: `19-monitoring-metrics-alerting-and-promql/07-promql-essentials-rate-increase-aggr.md`

Foundational PromQL reasoning

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- rate/increase nuances; guarding divide-by-zero naïveté; aggregate windows consciously.

Source: `19-monitoring-metrics-alerting-and-promql/08-alerting-threshold-response-playbooks.md`

Alert design + humane paging philosophy

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Symptom-based vs cause-based alert tension; practise firing synthetic alert on disposable notifier routes only.

Source: `19-monitoring-metrics-alerting-and-promql/09-monitoring-guided-troubleshooting-labs.md`

Metrics-first triage loops

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Inject CPU/mem/latency fault classes map signals → hypotheses succinctly—not instant restart reflex.

Source: `19-monitoring-metrics-alerting-and-promql/10-integration-symfony-go-prometheus-alerts.md`

Stack rehearsal

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Tie Symfony+Go scraping + representative alert rules journaling correlation expectations.

Phase — 20-logging-correlation-and-loki-aggregation

Directory: `20-logging-correlation-and-loki-aggregation/`

Source: `20-logging-correlation-and-loki-aggregation/01-log-level-structure-and-context-fields.md`

Deliberate logging payloads

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Structured JSON-ish fields capturing service, severity, durations; avoid screams of useless ALL CAPS strings lacking identifiers.

Source: `20-logging-correlation-and-loki-aggregation/02-structured-application-logs-php-go.md`

Application JSON logging coherence

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Align field naming across Symfony + Go for joinable analytics downstream responsibly.

Source: `20-logging-correlation-and-loki-aggregation/03-correlation-and-request-narratives.md`

Correlation identifiers spanning hops

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Propagate inbound trace/correlation headers across synchronous calls—even before full tracing sophistication matures organically.

Source: `20-logging-correlation-and-loki-aggregation/04-loki-ingestion-retention-thinking.md`

Loki ingestion & slicing queries

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Label hygiene + query performance caution; articulate retention budgeting privacy obligations.

Source: `20-logging-correlation-and-loki-aggregation/05-log-query-and-pattern-mining-discipline.md`

Search tactics isolating anomalies

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Target ERROR/timeouts/auth classes methodically—not infinite scroll burnout.

Source: `20-logging-correlation-and-loki-aggregation/06-layered-infra-vs-app-log-correlation.md`

Which log layer first under ambiguity?

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Tie ingress/proxy/container/app logs sequentially narrating causal chain rehearsal.

Source: `20-logging-correlation-and-loki-aggregation/07-root-cause-narratives-from-log-evidence-alone.md`

RCA insisting on textual evidence artefacts

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Disable instinctive reflex restarts prior comprehension unless safety demands immediate containment ethically.

Source: `20-logging-correlation-and-loki-aggregation/08-integration-centralised-logging-drill.md`

Traefik Symfony Go Redis Postgres → Loki

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Unified structured payloads + correlators; scripted failure requiring log-only RCA documentation.

Phase — 21-distributed-tracing-with-opentelemetry

Directory: `21-distributed-tracing-with-opentelemetry/`

Source: `21-distributed-tracing-with-opentelemetry/01-trace-vs-log-vs-metric-splits.md`

When tracing answers *where latency hides*

Outcome mindset: **request narrative** fidelity across cooperating processes.

Practise focus

- Differentiate aggregates (metrics), signal-rich events (logs), segmented timelines (traces).
- Construct minimal multi-span story HTTP→controller→dependency honestly on lab timings.

Source: `21-distributed-tracing-with-opentelemetry/02-opentelemetry-collector-routing-overview.md`

OpenTelemetry ingestion plumbing

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Run collector ethically disposing dev-only exporters—not production secret sprawl defaults.

Source: `21-distributed-tracing-with-opentelemetry/03-instrument-symfony-request-path.md`

Symfony span coverage

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Mark controller vs outbound HTTP spans; induce cooperative slow path observing span elongation ethically.

Source: `21-distributed-tracing-with-opentelemetry/04-instrument-go-service-handlers.md`

Go handler & dependency spans

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Capture DB/external client child spans; amplify SQL latency ethically using disposable dataset only.

Source: `21-distributed-tracing-with-opentelemetry/05-context-propagation-across-php-go.md`

Header propagation choreography

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Guarantee single trace identifiers survive internal hop—not trace disappearance ghost stories blaming vendors simplistically.

Source: `21-distributed-tracing-with-opentelemetry/06-database-span-and-query-latency.md`

Datastore span discipline

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Attach query segments cautiously sanitising literals / PII echoes from statements logging policies.

Source: `21-distributed-tracing-with-opentelemetry/07-sampling-and-cost-reality-for-tracing-at-scale.md`

Sampling trade-offs

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Head-based vs tail-informed sampling philosophies high-level—not logging every span eternally fantasies realistically.

Source: `21-distributed-tracing-with-opentelemetry/08-span-led-root-cause-rehearsals.md`

Hypothesis narrowing via waterfall spans

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Correlate timeout stories discovering dominating span—not vague slowness hand-waving devoid of timelines.

Source: `21-distributed-tracing-with-opentelemetry/09-integration-multi-hop-tracing-drill.md`

Traefik Symfony Go Redis Postgres stack

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Full propagation + deliberate fault requiring trace-led RCA artefacts documented responsibly.

Phase — 22-unified-observability-incidents-and-slo-thinking

Directory: `22-unified-observability-incidents-and-slo-thinking/`

Source: `22-unified-observability-incidents-and-slo-thinking/01-three-pillar-signal-thinking.md`

Three-pillar signal thinking

Outcome mindset: Interpret metrics, logs, and traces as complementary—not interchangeable—during investigations.

Practise focus

- Articulate qualitative roles: compression vs narrative vs segmented latency timelines.
- Dry-run hypothetical API slowdown deciding which pillar you inspect first under ambiguity without dogma.

Source: `22-unified-observability-incidents-and-slo-thinking/02-grafana-multi-signal-dashboards.md`

Unified dashboards with Grafana

Outcome mindset: Bind Prometheus-series visualisation with correlated log/trace drill paths responsibly.

Practise focus

- Fight chart noise; optimise for on-call skim efficiency even in laboratories mirroring seriousness.
- Tie dashboard variables to sane label hygiene preventing accidental cardinal explosions.

Source: `22-unified-observability-incidents-and-slo-thinking/03-cross-signal-correlation-lab.md`

Cross-signal slow dependency laboratory

Outcome mindset: Produce synthetic datastore stall aligning metric rise, textual timeout evidence, elongated SQL-span waterfall.

Practise focus

- Maintain timestamp-aware timeline narrative capturing corroborating artefact excerpts redacted ethically.

Source: `22-unified-observability-incidents-and-slo-thinking/04-incident-investigation-playbook-structure.md`

Structured incident rehearsal

Outcome mindset: Defer restart reflex until hypothesis unless immediate safety containment demands swifter blunt action.

Practise focus

- Capture impact, hypotheses, validations, corrective actions—even for simulated drills.

Source: `22-unified-observability-incidents-and-slo-thinking/05-service-health-using-golden-signals.md`

Comparative service health vistas

Outcome mindset: Observe Symfony versus Go workloads under identical synthetic load ethically scaled to lab realism.

Practise focus

- Explicitly annotate saturation nuances (connections, goroutines, thread pools proxies) verbally.

Source: `22-unified-observability-incidents-and-slo-thinking/06-slo-error-budget-and-sla-awareness.md`

SLO/error budget conversational literacy

Outcome mindset: Discuss availability targets aligning engineering trade-offs vs marketing SLA promises cautiously nuanced.

Practise focus

- Sketch qualitative error budgets without implying financial SLA guarantees absent business context.

Source: `22-unified-observability-incidents-and-slo-thinking/07-deep-span-log-metric-rca.md`

Multi-signal root cause deepening

Outcome mindset: Orchestrate complex fault requiring intertwined telemetry signals isolating bottleneck candidly documenting prevention backlog deltas.

Practise focus

Source: `22-unified-observability-incidents-and-slo-thinking/08-capstone-observability-chaos-integration.md`

Full-stack telemetry integration capstone

Outcome mindset: Traefik Symfony Go Redis Postgres + Prometheus Loki OTel Grafana; rotate chaos degradations journaling succinct postmortem-style artefacts.

Practise focus

Phase — 23-secrets-management-and-credential-security

Directory: `23-secrets-management-and-credential-security/`

Source: `23-secrets-management-and-credential-security/01-classify-production-secrets-vs-config.md`

Classify secrets vs benign config

Outcome mindset: Enumerate credential classes (DB, JWT, cloud keys, TLS private keys) responsibly reviewing Symfony/Go projects without dumping values.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `23-secrets-management-and-credential-security/02-environment-isolation-dotenv-discipline.md`

Environment separation pitfalls

Outcome mindset: Demonstrate staged env separation guarding against leaking prod artefacts into commits or shared clipboards unknowingly.

Practise focus

- Discuss `.env*` layering cautions and secret managers preferred over everlasting root-owned plaintext.

Source: `23-secrets-management-and-credential-security/03-kubernetes-secret-mounting-model.md`

Kubernetes Secret mechanics realism

Outcome mindset: Articulate secrets as encoded-at-rest blobs still requiring RBAC vigilance—not magical encryption fantasies by default unmanaged.

Practise focus

- Mount selectively; forbid logging secret material accidentally during startup debugging.

Source: `23-secrets-management-and-credential-security/04-secret-rotation-drill.md`

Rotation choreography

Outcome mindset: Exercise rotating JWT issuer secret or DB credential sequences observing coordinated rollout dependencies responsibly on labs only.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `23-secrets-management-and-credential-security/05-encryption-boundaries-thinking.md`

At-rest versus in-flight encryption interplay

Outcome mindset: Discuss disk encryption TLS overlays complementing—not replacing—secret hygiene and least privilege thoughtfully.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `23-secrets-management-and-credential-security/06-vault-dynamic-credentials-intro.md`

Dynamic credential issuance concept

Outcome mindset: Understand short-lived leased credentials diminishing static password staleness ethically experimenting with OSS Vault sandbox optionally.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `23-secrets-management-and-credential-security/07-gitlab-ci-secret-variables-policy.md`

CI masking & protected tiers

Outcome mindset: Leverage masked/protected variables; rehearse diagnosing pipeline failures when secret injections missing—avoid printing secrets casually.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `23-secrets-management-and-credential-security/08-credential-incident-remediation-talkthrough.md`

Synthetic leak tabletop

Outcome mindset: Walk tabletop: leaked IAM key revocation ordering, auditing, communicator responsibilities high-level ethically.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `23-secrets-management-and-credential-security/09-integration-secret-delivery-chain-capstone.md`

End-to-end secret delivery narration

Outcome mindset: Compose GitLab protected variables hypothetical · cluster secret artefacts · Symfony Go consumption choreography without plaintext logging.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Phase — 24-incident-troubleshooting-scenario-labs

Directory: `24-incident-troubleshooting-scenario-labs/`

Source: `24-incident-troubleshooting-scenario-labs/01-kubernetes-database-visibility-scenarios.md`

Kubernetes cannot reach database investigations

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Trace Service selector omissions, DNS flake, NetworkPolicy omission, Secrets/Config discrepancies ethically simulating disruptions disposable clusters responsibly.

Source: `24-incident-troubleshooting-scenario-labs/02-pod-instability-crashloops-resource-pressure.md`

Pod restart storms

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Correlate OOMKill vs failing readiness probes vs missing Secret volumes ethically rehearsing kubectl describe narratives responsibly.

Source: `24-incident-troubleshooting-scenario-labs/03-hot-cpu-hot-memory-metrics-led-triage.md`

Resource contention triage loops

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Refuse instantaneous restart urge; escalate through metrics/logs/traces alignment ethically synthetic load generation labs only responsibly.

Source: `24-incident-troubleshooting-scenario-labs/04-networking-tls-dns-ingress-502-chains.md`

Edge + mesh networking blackout rehearsals

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Practise TLS chain validation, DNS resolution failures, Ingress misroutes producing 502 ladders ethically documenting hypotheses responsibly.

Source: `24-incident-troubleshooting-scenario-labs/05-gitlab-runner-and-image-pipeline-incidents.md`

Pipeline failure forensics

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Diagnose brittle secrets, degraded runners, Dockerfile cache regressions ethically resisting blind rebuild loops sixfold irresponsibly.

Source: `24-incident-troubleshooting-scenario-labs/06-database-and-cache-timeout-scenarios.md`

Datastore latency incident simulations

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Correlate connection pool starvation vs slow queries vs saturation misconfiguration responsibly synthetic fault injections disposable databases ethically.

Source: `24-incident-troubleshooting-scenario-labs/07-full-stack-chaos-integration-doc.md`

Full stack chaos journaling

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Compose week synthesising rotational faults demanding structured evidence→hypothesis→mitigation artefacts responsibly ethically.

Phase — 25-performance-bottleneck-and-runtime-optimization

Directory: `25-performance-bottleneck-and-runtime-optimization/`

Source: `25-performance-bottleneck-and-runtime-optimization/01-performance-instrumentation-before-optimisation.md`

Measure-before-change discipline

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Enumerate latency vs throughput distinctions emphasising forbidding optimisation theatre absent baselines ethically.

Source: `25-performance-bottleneck-and-runtime-optimization/02-symfony-profiler-query-explosion-awareness.md`

Symfony Profiler & Doctrine query counts

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Demonstrate hypothetical N+1 detection remediation responsibly measuring before/after query counts ethically on disposable fixtures.

Source: `25-performance-bottleneck-and-runtime-optimization/03-go-pprof-cpu-memory-goroutines.md`

Go profiling surfaces

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Operate pprof ethically capturing goroutine/leak illustrative workloads annihilated afterwards responsibly avoiding production surprises.

Source: `25-performance-bottleneck-and-runtime-optimization/04-redis-caching-tradeoffs-invalidations.md`

Redis cache coherence discipline

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Articulate TTL invalidation pitfalls cache stampede mitigations ethically synthetic load glimpses labs only responsibly.

Source: `25-performance-bottleneck-and-runtime-optimization/05-database-connection-pool-tuning-shape.md`

Connection pool starvation narratives

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Tune Postgres pool boundaries observing exhaustion timeouts ethically restrained concurrency ramps laboratories responsibly.

Source: `25-performance-bottleneck-and-runtime-optimization/06-postgresql-slow-query-analysis.md`

Postgres latency & planning awareness

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Use EXPLAIN/ANALYZE patterns on disposable data; correlate with application query shapes responsibly.

Source: `25-performance-bottleneck-and-runtime-optimization/07-synthetic-load-k6-or-apachebench.md`

Load testing introductions

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Ramp concurrency ethically on lab stacks; observe p95 latency + error interplay without DDoSing shared networks.

Source: `25-performance-bottleneck-and-runtime-optimization/08-integration-bottleneck-hunt-marathon.md`

Cross-signal bottleneck capstone lab

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Combine profiler hints, Prometheus series, spans to isolate dominating constraint ethically rotating fault injections.

Source: `25-performance-bottleneck-and-runtime-optimization/09-postgres-operations-vacuum-and-stats-sketch.md`

Operations extensions for Postgres operators

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Touch autovacuum & pg_stat_statements narratives at high altitude deferring encyclopaedic mastery consciously.

Source: `25-performance-bottleneck-and-runtime-optimization/10-redis-production-operational-cautions.md`

Redis memory/eviction operational literacy

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Discuss persistence modes thundering herds mitigation responsibly illustrative synthetic mis-TTL rehearsals reversible ethically.

Source: `25-performance-bottleneck-and-runtime-optimization/11-async-queue-shape-and-worker-failure.md`

Queue systems failure literacy

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Outline DLQ retries idempotency using RabbitMQ/redis-stream parallels responsibly high-level illustrative labs ethically.

Source: `25-performance-bottleneck-and-runtime-optimization/12-database-centric-incident-scenarios-from-performance-view.md`

DB incidents bridging ops performance curricula

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Simulate deadlock/lock contention exhaustion tying metrics/logs/traces ethically disposable databases only conscientiously conscientiously ethically.

Source: `25-performance-bottleneck-and-runtime-optimization/13-async-queue-incident-drills.md`

Queue backlog escalation investigations

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Observe stalled consumers backlog growth alerting loops responsibly ethically lab poison messages eradicated afterwards thoughtfully.

Source: `25-performance-bottleneck-and-runtime-optimization/14-performance-capstone-integrated-stack.md`

Performance optimisation capstone rehearsal

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Traefik Symfony Go Redis Postgres Prometheus stack degrade→measure→repair journaling responsibly ethically.

Phase — 26-platform-engineering-and-internal-developer-platforms

Directory: `26-platform-engineering-and-internal-developer-platforms/`

Source:
`26-platform-engineering-and-internal-developer-platforms/01-internal-developer-platform-problem-shape.md`

Why platform teams mitigate ticket queues

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Articulate alleviating fragmented bespoke infra setups duplicating latent risk exponentially organisationally ethically.

Source: `26-platform-engineering-and-internal-developer-platforms/02-golden-paths-vs-chaos-custom-deploys.md`

Standard paved roads coexist with bounded flexibility

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Contrast fifty snowflake deployments vs opinionated templated corridors lowering cognitive load ethically thoughtful governance interplay.

Source: `26-platform-engineering-and-internal-developer-platforms/03-self-service-provisioning-fiction-to-practice.md`

Self-service infra narrative responsibly incremental

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Sketch GitLab-triggered provisioning reducing wait-time ethically acknowledging guardrails prerequisites observability hooks mandatory ethically.

Source: `26-platform-engineering-and-internal-developer-platforms/04-banish-eternal-console-clickops-default.md`

Automation default posture

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Advocate IaC/GitOps pipelines superseding brittle manual consoles except rare break-glass situations documented ethically conscientiously ethically.

Source: `26-platform-engineering-and-internal-developer-platforms/05-composable-modules-pipeline-templates.md`

Reusable modules & Helm/Terraform scaffolding

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Package network/compute Helm baseline templates ethically harmonising divergence consciously governance reviewed responsibly.

Source: `26-platform-engineering-and-internal-developer-platforms/06-service-catalog-abstraction-shape.md`

Service catalogue thinking

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Enumerate shared datastore cache queue secrets telemetry bundles developer selects responsibly reducing bespoke snowflakes ethically.

Source: `26-platform-engineering-and-internal-developer-platforms/07-platform-governance-guardrails-quality-bars.md`

Governance marrying enablement plus control

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Mandate telemetry security scanning ownership metadata baseline before production eligibility ethically harmonising organisational policy convergence responsibly.

Source: `26-platform-engineering-and-internal-developer-platforms/08-integration-paved-road-capstone-overview.md`

Paved-road integration storytelling

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Compose GitLab Dockerfile Helm Terraform Observability arcs minimising repetitive manual scaffolding ethically narrative capstone ethically deliberately conscientiously ethically.

Source: `26-platform-engineering-and-internal-developer-platforms/09-feedback-loops-platform-and-product.md`

Operational feedback bridging platform & product engineering

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Discuss SLO-informed prioritisation looping platform debt versus feature velocity tensions responsibly qualitatively ethically.

Source: `26-platform-engineering-and-internal-developer-platforms/10-deepening-reading-when-ready.md`

Defer deeper specialisation responsibly

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Pointer: revisit AWS/advanced Postgres/queues after practising depths earlier areas emphasising iterative deepening ethically consciously.
